

Universidade de Brasília – UnB

Faculdade de Ciências e Tecnologias em Engenharia – FCTE

Engenharia de *Software*

Focus: Desenvolvimento de um Aplicativo Gratuito e *Open Source* para Otimização do Aprendizado em Preparação para Provas

Autor: Artur Henrique Holz Bartz

Orientador: Dr. Wander Cleber Maria Pereira da Silva

Brasília, DF

2026



Artur Henrique Holz Bartz

Focus: Desenvolvimento de um Aplicativo Gratuito e *Open Source* para Otimização do Aprendizado em Preparação para Provas

Monografia submetida ao curso de graduação em Engenharia de *Software* da Universidade de Brasília, como requisito parcial para obtenção do Título de Bacharel em Engenharia de *Software*.

Universidade de Brasília – UnB

Faculdade de Ciências e Tecnologias em Engenharia – FCTE

Orientador: Dr. Wander Cleber Maria Pereira da Silva

Brasília, DF

2026

Artur Henrique Holz Bartz

Focus: Desenvolvimento de um Aplicativo Gratuito e *Open Source* para Otimização do Aprendizado em Preparação para Provas/ Artur Henrique Holz Bartz. – Brasília, DF, 2026-

87 p. : il. (algumas color.) ; 30 cm.

Orientador: Dr. Wander Cleber Maria Pereira da Silva

Trabalho de Conclusão de Curso – Universidade de Brasília – UnB
Faculdade de Ciências e Tecnologias em Engenharia – FCTE , 2026.

1. otimização de aprendizado. 2. gamificação. I. Dr. Wander Cleber Maria Pereira da Silva. II. Universidade de Brasília. III. Faculdade de Ciências e Tecnologias em Engenharia. IV. Focus: Desenvolvimento de um Aplicativo Gratuito e *Open Source* para Otimização do Aprendizado em Preparação para Provas

CDU [CÓDIGO CDU]

Artur Henrique Holz Bartz

Focus: Desenvolvimento de um Aplicativo Gratuito e *Open Source* para Otimização do Aprendizado em Preparação para Provas

Monografia submetida ao curso de graduação em Engenharia de *Software* da Universidade de Brasília, como requisito parcial para obtenção do Título de Bacharel em Engenharia de *Software*.

Trabalho apresentado. Brasília, DF, .

Dr. Wander Cleber Maria Pereira da Silva
Orientador

Convidado 1

Convidado 2

Brasília, DF
2026

Resumo

A preparação para avaliações acadêmicas exige do estudante universitário o domínio de estratégias de aprendizado eficazes e a atenção ao bem-estar mental. Porém, os *softwares* disponíveis no mercado são fragmentados, o que dificulta o acesso a um único aplicativo capaz de auxiliá-lo de forma integrada. Este trabalho apresenta o desenvolvimento de um aplicativo *mobile* gratuito e *open source* que integra um *Timer* Pomodoro, *flashcards* com repetição espaçada, gamificação e *check-ins* de humor. A metodologia adotada combinou principalmente o *framework Lean Inception* para o alinhamento da visão de produto e a definição de um escopo de Produto Mínimo Viável (MVP). Diante disso, foi realizado o levantamento de requisitos a partir de proto-personas, jornadas de usuário e *brainstorming* de funcionalidades, além da definição da arquitetura, do modelo de dados e de medidas de segurança e privacidade alinhadas à Lei Geral de Proteção de Dados (LGPD). O referencial teórico fundamenta as funcionalidades implementadas com base em evidências acadêmicas sobre gestão do tempo, repetição espaçada, gamificação e aprendizagem autorregulada. Como resultado, este trabalho entrega um aplicativo funcional, teoricamente fundamentado e centrado no usuário, incluindo a modelagem de dados, os diagramas de arquitetura e a implementação das funcionalidades priorizadas no MVP.

Palavras-chave: aplicativo *mobile*; gestão do tempo; repetição espaçada; gamificação; aprendizagem autorregulada.

Abstract

As a university student, preparing for academic tests demands the mastery of effective learning strategies and attention to mental well-being. However, currently available software solutions are fragmented, making it difficult for students to access a single application that supports them throughout this process. This work presents the development of a free and open-source mobile application that integrates a Pomodoro timer, spaced-repetition flashcards, gamification, and mood check-ins. The methodology adopted primarily combined the Lean Inception framework for product vision alignment and the definition of a Minimum Viable Product (MVP) scope. Accordingly, requirements were gathered through proto-personas, user journeys, and feature brainstorming, alongside the definition of the system architecture, data model, and privacy and security measures aligned with Brazil's General Data Protection Law (LGPD). The literature review grounds the implemented features in academic evidence regarding time management, spaced repetition, gamification, and self-regulated learning. As a result, this work delivers a functional, theoretically grounded, user-centered application, encompassing data modeling, architecture diagrams, and the implementation of the features prioritized in the MVP.

Key-words: mobile application; time management; spaced repetition; gamification; self-regulated learning.

Lista de ilustrações

Figura 1 – Tela inicial do Aprovado 26
Figura 2 – Tela inicial do StudyTok 27
Figura 3 – Tela inicial do Anki (AnkiDroid) 28
Figura 4 – Tela inicial do Habitica 29
Figura 5 – Tela inicial do Koki *FlashCards* 30
Figura 6 – Raia do TCC1 do BPMN do Processo Metodológico. 39
Figura 7 – Raia do TCC2 do BPMN do Processo Metodológico. 39
Figura 8 – BPMN do Processo Metodológico 40
Figura 9 – Ciclo de desenvolvimento TDD 42
Figura 10 – BPMN *Sprint* 44
Figura 11 – BPMN Kanban 45
Figura 12 – Quadro do *Brainstorming* de Funcionalidades 54
Figura 13 – Gráfico do Semáforo 56
Figura 14 – Sequenciador 58
Figura 15 – *Canvas* MVP 59
Figura 16 – Diagrama de Classes 74
Figura 17 – Diagrama de Pacotes do *Backend* 75
Figura 18 – Diagrama de Pacotes do *Frontend* 75
Figura 19 – Diagrama Entidade-Relacionamento (DE-R) do Projeto 76
Figura 20 – Diagrama Lógico do Projeto 77

Lista de tabelas

Tabela 1 – Comparativo entre o Focus e os concorrentes analisados 31
Tabela 2 – Jornada de Icarus 51
Tabela 3 – Jornada de Hazel 52
Tabela 4 – Jornada de Arabella 53
Tabela 5 – Requisitos Funcionais 65
Tabela 6 – Matriz de Rastreabilidade entre Histórias de Usuário e Requisitos Funcionais 66
Tabela 7 – Requisitos Não Funcionais 67

Lista de abreviaturas e siglas

5W1H	*What, Who, When, Where, Why, How*
ABP	Aprendizagem Baseada em Problemas
AI	*Artificial Intelligence*
APIs RESTful	*Application Programming Interfaces Representational State Transfer*
ARE	Algoritmo de Repetição Espaçada
BPMN	*Business Process Model and Notation*
CABCM	*Concept Association-Based Concept Mapping*
COVID-19	*Corona Virus Disease 2019*
CRUD	*Create, Read, Update, Delete*
DE-R	Diagrama Entidade-Relacionamento
EFL	*English as a Foreign Language*
FCTE	Faculdade de Ciências e Tecnologias em Engenharia
FURPS	*Functionality, Usability, Reliability, Performance, Supportability*
GBL	*Game-Based Learning*
HTTP	*HyperText Transfer Protocol*
HTTPS	*HyperText Transfer Protocol Secure*
IA	Inteligência Artificial
JSON	*JavaScript Object Notation*
JWT	*JSON Web Token*
LASSI	*Learning and Study Strategies Inventory*
LGPD	Lei Geral de Proteção de Dados
MoSCoW	*Must have, Should have, Could have, Won't have*
MVP	*Minimum Viable Product*

ORM	*Object Relational Mapping*
PDF	*Portable Document Format*
SGBDR	Sistema Gerenciador de Banco de Dados Relacional
SRL	*Self-Regulated Learning*
SRS	*Spaced Repetition System*
TCC	Trabalho de Conclusão de Curso
TLS	*Transport Layer Security*
UML	*Unified Modeling Language*
US	*User Story*
UX	*User Experience*
XP	*Extreme Programming*

Sumário

1 INTRODUÇÃO . . . 12
1.1 Contextualização . . . 13
1.2 Justificativa . . . 14
1.3 Questão de Pesquisa . . . 15
1.4 Objetivos . . . 15
1.4.1 Objetivo Geral . . . 15
1.4.2 Objetivos Específicos . . . 15
1.5 Organização do Trabalho . . . 16

2 REFERENCIAL TEÓRICO . . . 17
2.1 Palavras-chave . . . 17
2.2 Rastros das Consultas . . . 18
2.3 Gestão do Tempo . . . 22
2.4 Metodologias de Aprendizado . . . 23
2.5 Gamificação . . . 24
2.6 Aplicativos Similares . . . 25
2.7 Heurísticas de Nielsen . . . 32
2.8 Algoritmo de Repetição Espaçada (ARE) . . . 32
2.9 Pomodoro *Timer* . . . 33
2.10 Aprendizagem Autorregulada (*Check-ins* de Humor) . . . 33
2.11 Autossabotagem e Adesão ao Uso do Aplicativo . . . 34

3 METODOLOGIA . . . 36
3.1 Processo Metodológico . . . 36
3.1.1 TCC1 e TCC2 . . . 36
3.1.2 TCC2 . . . 39
3.2 Metodologia de Desenvolvimento . . . 40
3.2.1 Metodologia Ágil . . . 41
3.2.2 *Extreme Programming* (XP) . . . 41
3.2.3 *Test-Driven Development* . . . 42
3.2.4 *Scrum* . . . 43
3.2.5 Kanban . . . 44
3.2.6 *Lean Inception* . . . 45
3.3 Requisitos . . . 59
3.3.1 Requisitos Funcionais . . . 60
3.3.2 Requisitos Não Funcionais . . . 66

4 DESENVOLVIMENTO 68
4.1 Suporte Tecnológico 68
4.1.1 Miro 68
4.1.2 Figma 68
4.1.3 Draw.io 68
4.1.4 React Native e Expo 69
4.1.5 Node.js e Express 69
4.1.6 PostgreSQL 69
4.1.7 Git, GitHub e Licenciamento 69
4.1.8 Trello 70
4.1.9 BrModelo 70
4.1.10 Claude (Assistente de IA) 70
4.2 Arquitetura 71
4.2.1 Padrão Arquitetural em Camadas 71
4.2.2 *Backend* 71
4.2.3 API RESTful, Segurança e Privacidade 72
4.2.4 Mapeamento Objeto-Relacional (ORM) 72
4.2.5 *Frontend* 73
4.3 Representação Visual do Sistema 73
4.3.1 Diagrama de Classes 73
4.3.2 Diagrama de Pacotes 74
4.4 Projeto de Dados 75
4.4.1 Diagrama Conceitual 76
4.4.2 Diagrama Lógico 76
4.5 Testes de Software 77
4.6 Prototipação 78
4.7 Implantação em Produção 79
4.7.1 Banco de Dados em Nuvem 79
4.7.2 Hospedagem do *Backend* 79
4.7.3 Geração e Distribuição do Aplicativo 80

5 CONSIDERAÇÕES FINAIS 82
5.1 Conclusão 82
5.2 Trabalhos Futuros 83

REFERÊNCIAS 84

1 Introdução

A preparação para provas acadêmicas caracteriza-se como um processo intenso, em que apenas a dedicação não garante o sucesso. A jornada do estudante universitário é marcada pela necessidade de dominar grandes quantidades de conteúdo e de converter esse conhecimento em resultados sob a pressão do exame. Dessa maneira, o tema deste trabalho é o desenvolvimento de uma tecnologia que engloba a gestão do processo de estudo e o bem-estar do usuário. A eficácia na preparação para provas exige que o estudante demonstre habilidades na gestão do seu tempo de estudos e habilidades em lidar com a ansiedade causada pela avaliação. A competência em provas e a gestão do tempo são um dos fatores mais correlatos com a performance acadêmica (KLEIJN; PLOEG; TOPMAN, 1994), evidenciando que a falha em qualquer um desses componentes é uma causa de baixo rendimento.

As causas do baixo rendimento na preparação para provas podem ser explicadas pelos hábitos de estudo ineficientes. Muitos estudantes deixam de lado o valor da prática de recuperação ativa e da autoavaliação contínua, utilizando estratégias passivas que não trazem sucesso na hora da prova. O desenvolvimento de um alto desempenho está ligado à execução de estratégias de estudo específicas. Por exemplo, estudos que utilizam o inventário LASSI (*Learning and Study Strategies Inventory*) apontaram que os componentes de estratégias de prova e concentração são significativos para um bom rendimento (ZHOU; GRAHAM; WEST, 2016). Portanto, a ausência de sistemas de repetição espaçada de conteúdos e cronogramas bem definidos são causas da ineficiência do estudo.

Como consequência do estudo desorganizado e da ansiedade trazida pelas avaliações, os estudantes enfrentam uma elevada pressão psicológica. A gestão ineficiente do tempo e a baixa utilização das estratégias de estudo resultam em altos níveis de ansiedade e estresse. Esse quadro psicológico tem um efeito prejudicial no desempenho e no bem-estar do estudante. Dessa maneira, a autorregulação e a resiliência são destacadas como dimensões que mitigam esses desafios, sugerindo que a preparação não pode ignorar a saúde mental do indivíduo (LIU; LU; YAN, 2025).

Diante disso, o mercado de aplicativos *mobile* de estudo tem crescido, mas de forma segmentada. Existem aplicativos dedicados à gestão do tempo, à repetição espaçada de conteúdos e à motivação, porém sempre de forma unicamente dedicada àquele componente. Contudo, essa fragmentação exige que o usuário integre manualmente diferentes aplicativos a sua rotina, prejudicando a eficácia e a análise geral do progresso. Portanto, a oportunidade de inovação surge na unificação dessas funcionalidades em um único aplicativo, além de trazer novos campos como a gamificação e análises de estresse.

A necessidade de incluir a gamificação é incentivada pela comunidade acadêmica, que otimiza o aprendizado e o aumento do engajamento (MARENGO et al., 2025).

Portanto, este trabalho preenche essa lacuna ao desenvolver um aplicativo integrado e eficiente. Dessa maneira, o trabalho apresenta o desenvolvimento de um aplicativo *mobile*, *open source* e gratuito para otimização do aprendizado acadêmico, com o objetivo de integrar as metodologias de estudo mais validadas pela comunidade acadêmica em uma única aplicação, já que as estratégias de aprendizado e os hábitos de estudo estão diretamente ligados ao desempenho acadêmico (BICKERDIKE et al., 2016). Assim, o presente trabalho desenvolve a engenharia de *software* de uma ferramenta que atende à demanda por uma preparação eficaz, organizada e psicologicamente saudável.

1.1 Contextualização

A preparação para avaliações está presente no campo da educação e da qualidade de vida das pessoas, onde o desempenho está relativamente ligado à profissão que esse indivíduo pode alcançar. O estudante, muitas vezes sobrecarregado, precisa converter tempo de estudo em retenção a longo prazo e bom desempenho nessas avaliações. Esse cenário competitivo exige que o candidato desenvolva estratégias de aprendizagem e hábitos de estudo a fim de alcançar bons resultados. Abordagens focadas em habilidades, e não apenas em conteúdo, são suportadas por estudos que confirmam uma associação entre as estratégias de aprendizado, os hábitos de estudo e o desempenho acadêmico (BICKERDIKE et al., 2016). Portanto, o desenvolvimento de uma ferramenta deve ter como foco principal a otimização dessas metodologias.

Um dos desafios que a preparação para provas traz é a gestão da ansiedade e o foco. O estudo desorganizado e a sensação de perda de controle sobre o volume de material são fatores que intensificam o estresse. Embora o efeito prejudicial da ansiedade no desempenho seja um fenômeno documentado, a comunidade acadêmica sugere que para melhorar o desempenho deve-se focar nas causas dessa ansiedade, que geralmente estão ligadas à falta de técnicas de estudo adequadas (KLEIJN; PLOEG; TOPMAN, 1994). Isso justifica a necessidade de o aplicativo integrar recursos de foco, como um *timer* cíclico de sessões de estudo, e de gráficos de níveis de estresse ao longo do tempo, com o objetivo de apoiar o planejamento das sessões e possibilitar uma análise geral do bem-estar do estudante, respectivamente.

O sucesso na prova não vem apenas da quantidade de informações absorvidas, mas sim da eficiência em recuperá-las com qualidade. A eficácia das técnicas de estudo propostas no aplicativo, como *flashcards* com repetição espaçada, é comprovada pela sua capacidade de forçar a recuperação ativa. Ao analisar os fatores de desempenho, foi de-

monstrado que o gerenciamento do tempo e a autoavaliação são fatores fortes no desempenho acadêmico (ZHOU; GRAHAM; WEST, 2016). Isso valida o foco em funcionalidades que promovam a prática constante de recuperação de informações, elemento central na autorregulação e na retenção de informação.

Finalmente, a qualidade da preparação para provas depende da atenção ao bem-estar e à saúde mental. O estudante deve ser tratado como um indivíduo que precisa de suporte para lidar com o estresse. Os recursos de autorregistro de humor (*check-ins*) que o aplicativo oferece, a fim de gerar gráficos intuitivos, apoiam o estudante a perceber, ao longo do tempo, possíveis relações entre suas técnicas de estudo e seu nível de estresse. Estudos sobre essa área indicam que as três dimensões da resiliência na aprendizagem (autorregulação, sociabilidade e empatia) têm um efeito positivo direto no desempenho acadêmico (LIU; LU; YAN, 2025). Ao fornecer ferramentas para que o estudante monitore e gerencie sua autorregulação e seu estado mental, o aplicativo representa um facilitador de um estudo mais equilibrado e eficiente.

1.2 Justificativa

O desenvolvimento desse aplicativo se justifica pela necessidade de superar a ineficiência dos métodos de estudo passivos e desorganizados, que são normalmente adotados pelos estudantes. A dedicação à leitura de materiais não garante a retenção nem a aplicação do conhecimento em situações de avaliações. Nesse sentido, o trabalho propõe transformar o estudo em um processo ativo e estruturado, baseado em evidências da comunidade acadêmica. A importância dessa intervenção é reforçada pela pesquisa, que demonstra que o aprimoramento das habilidades de estratégias de estudo está associado ao sucesso acadêmico em provas (KHALIL; WILLIAMS; HAWKINS, 2024).

O principal princípio que o projeto corrige é a falta de um sistema integrado que promova a organização do estudante, atentando também à sua saúde mental. O sucesso acadêmico é influenciado por componentes estratégicos como habilidade, vontade e autorregulação, portanto as funcionalidades de gestão de tempo, cronômetro cíclico e autoavaliação do aplicativo apoiam esse último componente. A literatura, ao analisar o LASSI, especifica que as subescalas de gerenciamento do tempo e autoavaliação são fatores mais fortes no desempenho acadêmico em períodos iniciais (ZHOU; GRAHAM; WEST, 2016). Isso justifica a seleção destas funcionalidades como elementos centrais do *software*.

No mercado atual de ferramentas de otimização de estudo, as opções são normalmente fragmentadas. Existem aplicativos de cronômetros cíclicos, gerenciadores de tarefas e plataformas de *flashcards* com repetição espaçada, mas há a ausência de uma plataforma unificada que colete dados de todas essas interações, impedindo uma visão mais completa

do progresso. A justificativa deste trabalho está na unificação dessas áreas com a atenção ao bem-estar do estudante, contendo *check-ins* de humor para o autorregistro do nível de ansiedade e estresse percebidos. Outra adição do aplicativo é a gamificação, que é uma estratégia validada para a otimização do aprendizado e aumento do engajamento (MARENGO et al., 2025).

Finalmente, o desenvolvimento deste trabalho representa uma contribuição ideal para a área de engenharia de *software* e educação. Ao projetar uma engenharia de *software* em um aplicativo *open source* e gratuito, o trabalho democratiza o acesso a metodologias de estudo além de disponibilizar um código fonte estruturado conforme as normas da engenharia de *software* para o aprendizado de qualquer pessoa. Portanto, este projeto é uma aplicação socialmente relevante da ciência da computação para a otimização educacional.

1.3 Questão de Pesquisa

Para fundamentar o desenvolvimento deste trabalho, a questão de pesquisa foi formalmente definida: Como desenvolver um aplicativo móvel integrando ciclos de foco, repetição espaçada, gamificação e autorregistro de bem-estar para apoiar a organização da rotina de estudos de estudantes em preparação para avaliações?

1.4 Objetivos

Este trabalho possui um objetivo geral e outros cinco objetivos específicos que serão aprofundados nesta seção.

1.4.1 Objetivo Geral

Este trabalho possui como objetivo geral desenvolver e avaliar um aplicativo móvel gratuito e de código aberto que integre técnicas de gestão do tempo, repetição espaçada, gamificação e autorregistro de bem-estar para apoiar a organização da rotina de estudos.

1.4.2 Objetivos Específicos

São objetivos específicos deste trabalho:

- Identificar, na literatura e junto a potenciais usuários, necessidades relacionadas à organização dos estudos;

- Definir método de priorização de requisitos funcionais e não funcionais para um MVP;
- Definir um método de desenvolvimento da solução;
- Projetar a arquitetura, o modelo de dados e as interfaces do aplicativo;
- Implementar e validar as funcionalidades priorizadas.

1.5 Organização do Trabalho

O presente trabalho está dividido nos seguintes capítulos:

- **Capítulo 1 - Introdução:** A função do capítulo introdutório é preparar o leitor ao descrever o cenário em que o aplicativo foi desenvolvido, apresentar o porquê da pesquisa, mostrar a pergunta-chave do trabalho e resumir as estratégias empregadas.
- **Capítulo 2 - Referencial Teórico:** Dividido em tópicos, este capítulo desenvolve a base teórica do estudo, introduzindo conceitos importantes relacionados ao tema. Os tópicos abordados englobam desde a gestão de tempo e metodologias de estudo até a gamificação, além de uma revisão sobre aplicativos já existentes.
- **Capítulo 3 - Metodologia:** Neste capítulo, é detalhada a metodologia de pesquisa e desenvolvimento do projeto. Diante disso, são explicadas as abordagens que apoiaram a implementação da ferramenta de *software*.
- **Capítulo 4 - Desenvolvimento:** O capítulo aprofunda os detalhes da fase de desenvolvimento do *software*, incluindo as otimizações e modificações realizadas para assegurar uma implementação de sucesso.
- **Capítulo 5 - Considerações Finais:** O capítulo final resume os resultados obtidos pelo trabalho e indica sugestões para projetos ou estudos subsequentes.

2 Referencial Teórico

O referencial teórico que sustenta este trabalho é explorado neste capítulo. Para isso, foram feitas consultas de literatura utilizando *strings* de busca na base científica Scopus, com foco nos principais conceitos que sustentam o tema deste trabalho. A seguir estão listadas as palavras-chave e os rastros das consultas realizadas com elas:

2.1 Palavras-chave

Português:

- Otimização de Aprendizado
- Gestão do Tempo
- Pomodoro
- Gamificação
- Cartões de Aprendizado
- Psicologia
- Motivação
- Estresse
- Ansiedade
- Metodologias de Aprendizado

Inglês:

- *Learning Optimization*
- *Time Management*
- *Pomodoro Technique*
- *Gamification*
- *Flashcards*
- *Cards*

- *Psychology*
- *Motivation*
- *Stress*
- *Anxiety*
- *Learning Methodologies*

2.2 Rastros das Consultas

Usando a *string* de busca ("Learning Optimization"AND ("Gamification"OR "Spaced Repetition"OR "Pomodoro Technique")) foram retornados quatro artigos. Lendo os títulos destes, foi escolhido o artigo de título:

- *Research AI: integrating AI and gamification in higher education for e-learning optimization and soft skills assessment through a cross-study synthesis*

Usando a *string* de busca ("Time Management"AND "Academic Performance"AND "Study Strategy") foram retornados dezenove artigos. Lendo os títulos destes, foram escolhidos os artigos de títulos:

- *Social Skills and Stress among Psychology, Nursing and Medical Students*
- *Learning and study strategies correlate with medical students' performance in anatomical sciences*
- *The relationship between study strategies and academic performance*
- *Learning strategies, study habits and social networking activity of undergraduate medical students*
- *Cognition, study habits, test anxiety, and academic performance*

Usando a *string* de busca ("Psychology"AND ("Stress"OR "Anxiety") AND "Student"AND "Academic Achievement") foram retornados novecentos e três artigos. Lendo os títulos dos vinte primeiros, foi escolhido o artigo de título:

- *Exploring the mediating role of anxiety between resilience and academic achievement in students' English learning*

Usando a *string* de busca ("Time Management") foram retornados quinze mil quinhentos e quarenta artigos. Lendo os títulos dos sessenta primeiros, foi escolhido o artigo de título:

- *Unlocking academic success: the impact of time management on college students' study engagement*

Usando a *string* de busca ("Time Management"OR "Time Planning"OR "Time Management Skills") foram retornados dezenove mil duzentos e vinte artigos. Lendo os títulos dos sessenta primeiros, foi escolhido o artigo de título:

- *Improving medical students' learning strategies, management of workload and well-being: a mixed methods case study in undergraduate medical education*

Usando a *string* de busca ("learning methodology"OR "teaching method"OR "pedagogical approach") foram retornados sessenta e sete mil cento e quatorze artigos. Lendo os títulos dos cinquenta primeiros, foram escolhidos os artigos de títulos:

- *Didactic methodologies used in police training schools to teach human rights*
- *A comprehensive framework for higher education 4.0*
- *Intelligent Technology-Driven Hybrid English Classroom Model for Enhanced Personalization and Learning Efficiency*

Usando a *string* de busca ("Gamification"OR "Gamified Learning"OR "Game-Based Learning") foram retornados vinte e nove mil oitocentos e vinte artigos. Lendo os títulos dos primeiros artigos, foram escolhidos os seguintes títulos:

- *Effects of game-based learning integrated with different thinking-guided methods on computational thinking of elementary school students*
- *Gamified feedback in adaptive retrieval practice: Points and progress-bars enhance motivation but not learning*

Usando a *string* de busca ("Spaced Repetition System") foram retornados treze artigos. Lendo todos os títulos, foi escolhido o seguinte título:

- *Investigating the use of a mobile flashcard application rememba on the vocabulary development and motivation of EFL learners*

Usando a *string* de busca ("self-regulated learning") foram retornados treze artigos. Lendo todos os títulos, foi escolhido o seguinte título:

- *Promoting self-regulated learning during the covid-mandated remote learning period: Insights from interviews with primary school teachers*

Usando a *string* de busca ("The Pomodoro Technique") foram retornados treze artigos. Lendo todos os títulos, foi escolhido o seguinte título:

- *Assessing the efficacy of the Pomodoro technique in enhancing anatomy lesson retention during study sessions: a scoping review*

Usando a *string* de busca ("software development methodologies") foram retornados mil setecentos e quarenta e um artigos. Lendo os títulos dos vinte primeiros, foi escolhido o seguinte título:

- *From Traditional to Agile Methodologies in Software Project Management Education: A Case Study*

Usando a *string* de busca ("Extreme Programming") foram retornados mil quinhentos e oitenta e dois artigos. Lendo os títulos dos vinte primeiros, foi escolhido o seguinte título:

- *Implementation of Extreme Programming Method in Developing Website of Agribusiness Harvest Transportation Order Data Management*

Usando a *string* de busca ("Scrum") foram retornados cinco mil setecentos e sessenta e sete artigos. Lendo os títulos dos vinte primeiros, foi escolhido o seguinte título:

- *Applying Scrum to Knowledge Transfer Among Software Developers*

Usando a *string* de busca ("Kanban") foram retornados dois mil seiscentos e quarenta e dois artigos. Lendo os títulos dos vinte primeiros, foi escolhido o seguinte título:

- *Effect of the Kanban Method on Problem Based Learning applied to university students*

Usando a *string* de busca ("Lean Inception") foram retornados doze artigos. Lendo todos os títulos, foi escolhido o seguinte título:

- *Investigating Problem Definition and End-User Involvement in Agile Projects that Use Lean Inceptions*

Usando a *string* de busca ("test-driven development") foram retornados mil cento e setenta e quatro artigos. Lendo os títulos dos dez primeiros, foi escolhido o seguinte título:

- *On the use of Test-Driven Development for Embedded Systems*

Usando a *string* de busca ("Software Requirements") foram retornados seis mil duzentos e noventa e quatro artigos. Lendo os títulos dos cinquenta e um primeiros, foi escolhido o seguinte título:

- *Ensuring quality in software requirement engineering process: A comparative study*

Usando a *string* de busca ("FURPS") foram retornados quarenta e dois artigos. Lendo os títulos dos dez primeiros, foi escolhido o seguinte título:

- *Software Quality Attributes in Requirements Engineering*

Usando a *string* de busca ("software architecture") foram retornados trinta e cinco mil cento e noventa e sete artigos. Lendo os títulos dos cento e trinta primeiros, foi escolhido o seguinte título:

- *Editorial of the special issue on Quality in Software Architecture*

Usando a *string* de busca ("UML") foram retornados vinte e três mil quatrocentos e sessenta e quatro artigos. Lendo os títulos dos oitenta primeiros, foi escolhido o seguinte título:

- *A Research on the Impact of a Class Diagram Layout and Complexity on System Model Understandability*

Usando a *string* de busca ("Entity-Relationship Model") com o filtro de acesso livre foram retornados cento e quarenta e sete artigos. Lendo os títulos dos dez primeiros, foi escolhido o seguinte título:

- *Difficulties in understanding the transition between database design stages: An experiment from a semantic distance perspective*

Usando a *string* de busca ("Logic Data Model") com o filtro de acesso livre foram retornados dois artigos. Lendo todos os títulos, foi escolhido o seguinte:

- *Applying model-driven approach to building rapid distributed data services*

Além disso, foram coletados, em buscas na internet, quatro livros e três artigos para dar base às referências teóricas deste trabalho. Seus títulos podem ser observados abaixo:

- *The Future of Software Quality Assurance*
- *Handbook of Software Engineering Methods*
- *Usability Engineering*
- *The Pomodoro Technique: The Life-Changing Time-Management System*
- *Optimization of repetition spacing in the practice of learning*
- *Choice overload: A conceptual review and meta-analysis*
- *When and why adults abandon lifestyle behavior and mental health mobile apps: Scoping review*

2.3 Gestão do Tempo

A gestão do tempo é um conceito fundamental no ambiente de preparação para provas, sendo definida como a disposição e a capacidade do indivíduo de planejar, agendar e executar atividades de estudo de forma eficaz. Essa competência se define na habilidade do estudante de estabelecer metas, priorizar tarefas e otimizar o uso do tempo disponível. Pesquisas demonstram que a disposição para o gerenciamento do tempo está diretamente associada ao engajamento nos estudos, sendo um fator importante para o sucesso acadêmico na jornada universitária (FU et al., 2025).

Diante disso, em estudos realizados com estudantes, o componente de gestão do tempo se mostrou como um forte fator no desempenho acadêmico (ZHOU; GRAHAM; WEST, 2016). Portanto, a eficácia na gestão do tempo está diretamente ligada ao desempenho. Isso sugere que o planejamento do tempo de estudo e o estabelecimento de rotinas são tão importantes para os resultados quanto as habilidades de autoavaliação e motivação.

Uma gestão de tempo eficiente contribui para o aumento do autocontrole do estudante, o que está ligado a um maior engajamento nos estudos (FU et al., 2025). Além

disso, a definição de rotinas estruturadas previnem o uso de dispositivos móveis durante sessões de estudo, reforçando a ideia de que o planejamento ajuda a mitigar distrações e a manter-se focado.

A transição para o ambiente universitário, que torna necessário o estudo independente, coloca a gestão da carga de trabalho e o uso estratégico do tempo no centro das estratégias de aprendizagem. Em cursos exigentes, a mudança dos métodos passivos de estudo para estratégias mais ativas exige que os estudantes aprimorem suas habilidades de autogestão (HARVEY et al., 2025). Assim, a gestão do tempo é uma estratégia de aprendizado que se torna necessária para que os alunos consigam adaptar-se à demanda da carga horária.

Finalmente, a gestão adequada do tempo se revela essencial para a manutenção do bem-estar e para a prevenção de sobrecarga mental de estudantes. A gestão da carga de trabalho, um componente do gerenciamento do tempo, é crucial para o bem-estar dos estudantes, especialmente em ambientes de alta pressão (HARVEY et al., 2025). Quando os alunos dominam técnicas de organização do tempo, torna-se possível distribuir suas tarefas de forma mais equilibrada, reduzindo a sensação de ansiedade e estresse.

2.4 Metodologias de Aprendizado

A evolução dos ambientes de ensino superior implica diretamente na forma de se ensinar, migrando de abordagens centradas no professor para modelos pedagógicos focados no estudante. Essa mudança é um dos pilares da chamada Educação Superior 4.0, que exige o desenvolvimento de competências, como a capacidade de resolução de problemas e o pensamento crítico (AMGHAR; BENCHEKROUN, 2025). Tais metodologias buscam promover o engajamento ativo e a autonomia da jornada educacional, preparando os indivíduos para um mercado de trabalho dinâmico.

As metodologias ativas surgem como ferramentas para aprimorar a eficiência do aprendizado. Um exemplo disso é o modelo de sala de aula híbrida impulsionado pela tecnologia, que visa otimizar o processo de ensino e fornecer suporte de aprendizagem (WANG; DU, 2026). Essas abordagens permitem que o conteúdo seja adaptado às necessidades e ao ritmo de cada aluno, superando as limitações dos métodos tradicionais de ensino presencial.

A escolha e a aplicação das metodologias didáticas devem ser adequadas ao conteúdo e aos objetivos, sendo importante a adoção de estratégias que estimulem a participação e a reflexão. Em contextos específicos, como o treinamento policial em direitos humanos, a comunidade acadêmica sugere que as metodologias mais utilizadas incluem o estudo de casos, as conferências com especialistas e os grupos focais, que visam a sensi-

bilização e a reflexão sobre valores éticos e princípios legais (FERNáNDEZ et al., 2026). A eficácia da metodologia está ligada à sua capacidade de mudar a técnica passiva do estudante para uma participação crítica no processo aprendizado.

2.5 Gamificação

A gamificação e a Aprendizagem Baseada em Jogos (*Game-Based Learning* - GBL) representam uma área de estudo crescente, sendo reconhecidas como ferramentas capazes de potencializar o envolvimento e o interesse. A GBL demonstrou ser uma metodologia eficaz para o desenvolvimento do pensamento computacional em estudantes do ensino fundamental (HSU; HSU, 2026). Esse tipo de abordagem transforma o processo de aprendizagem em uma experiência motivadora, integrando elementos de resolução de problemas por meio de cenários de jogo.

No entanto, o impacto da gamificação se manifesta majoritariamente na motivação, e normalmente não traz ganhos na aprendizagem em si. Elementos gamificados, como sistemas de pontuação e barras de progresso, são altamente eficazes para aumentar a motivação e a persistência dos estudantes em estratégias de estudo que exigem esforço, como a prática de recuperação adaptativa (BROEK et al., 2026). Contudo, essa motivação não garante um aumento nos resultados destes estudos, apenas age como um positivo adicional à aprendizagem.

Para maximizar os benefícios da gamificação, é necessário que ela seja integrada a métodos de aprendizado eficazes. A combinação do aprendizado baseado em jogos com métodos de orientação, como o 5W1H (o quê, quem, quando, onde, por que e como) ou o mapeamento de conceitos baseado em associação (CABCM), demonstrou ser eficaz na promoção do pensamento computacional e da autoeficácia de estudantes (HSU; HSU, 2026). Portanto, enquanto a gamificação fornece a estrutura motivacional, a inclusão de metodologias de aprendizado e de resolução de problemas é essencial para transformar esse engajamento em um resultado real.

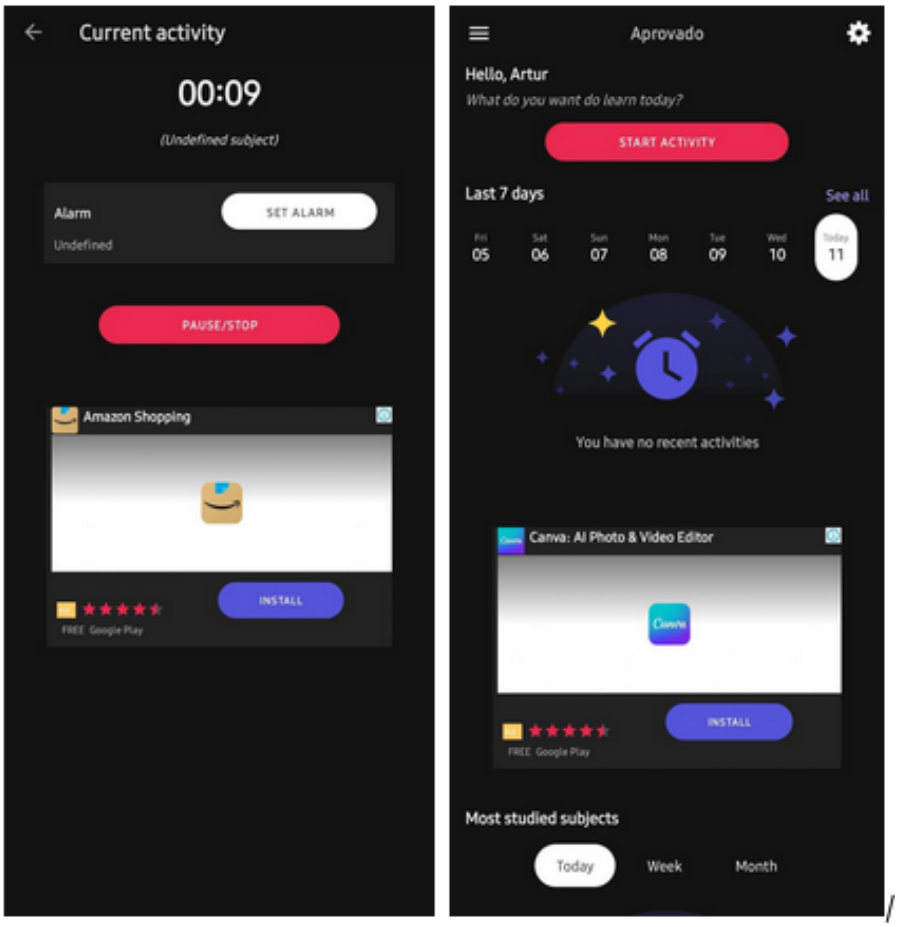
2.6 Aplicativos Similares

Foi feito um *benchmark* sobre aplicativos similares com objetivos de auxiliar na rotina de estudos dos estudantes.

Um deles é o Aprovado, mostrado na Figura 1, cujo seu propósito é atuar como uma ferramenta de produtividade e controle de tempo voltada para o estudante. Seu principal recurso é o gerenciamento das horas de estudo, permitindo que o usuário registre o tempo dedicado a cada matéria, gerando gráficos e relatórios sobre a evolução e o desempenho nessas matérias. Ao focar em métricas e estatísticas, o Aprovado se alinha com o pilar de gestão do tempo deste trabalho, fornecendo uma ideia de onde o tempo está sendo aplicado, o que é relevante para a organização da rotina de um estudante (APROVADO, 2026). Porém, o aplicativo não se concentra no campo da saúde mental do estudante e, logo, não traz uma visão da qualidade do estudo.

Dessa maneira, o Aprovado apresenta limitações ao ser comparado com o escopo do Focus, aplicativo desenvolvido neste trabalho. Enquanto o projeto oferece uma solução completa que integra gestão do tempo, prática de recuperação ativa, gamificação e coleta de dados de bem-estar, o Aprovado é majoritariamente uma ferramenta de controle de horas.

Figura 1 – Tela inicial do Aprovado

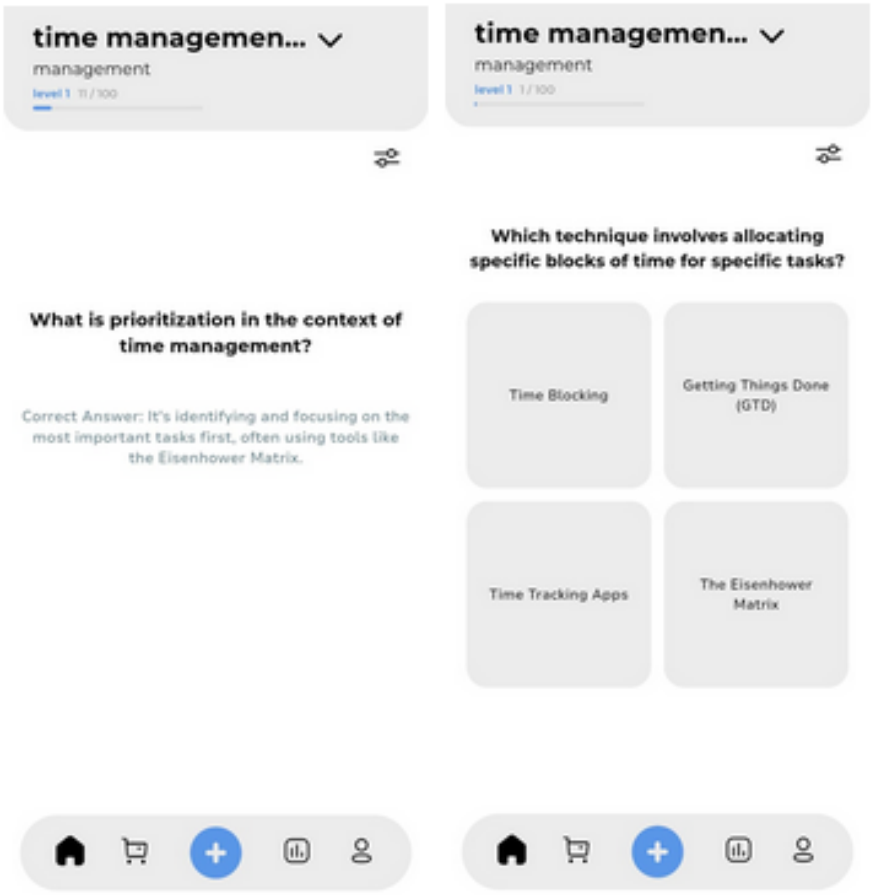


Fonte: (APROVADO, 2026).

De outra maneira, o StudyTok, mostrado na Figura 2, é uma plataforma mais dinâmica, utilizando a Inteligência Artificial (IA) para transformar materiais de estudo em componentes dentro da aplicação. Seu principal valor é a funcionalidade de geração de material de estudo, em que o usuário envia imagens, PDFs, PowerPoints e vídeos para o aplicativo e, em seguida a IA gera questionários e *flashcards*. Essa funcionalidade está diretamente ligada ao valor da prática de recuperação ativa, permitindo que a IA crie os recursos necessários para a repetição espaçada ao invés do próprio aluno, otimizando o tempo que seria gasto na criação manual (STUDYTOK, 2026).

Além disso, o aplicativo possui a gamificação através de placares de líderes e um sistema de recompensa por pontos a fim de manter a motivação do usuário. Portanto, em comparação com este trabalho, o StudyTok não possui os valores de gestão do tempo e controle da saúde mental que este trabalho implementa (STUDYTOK, 2026).

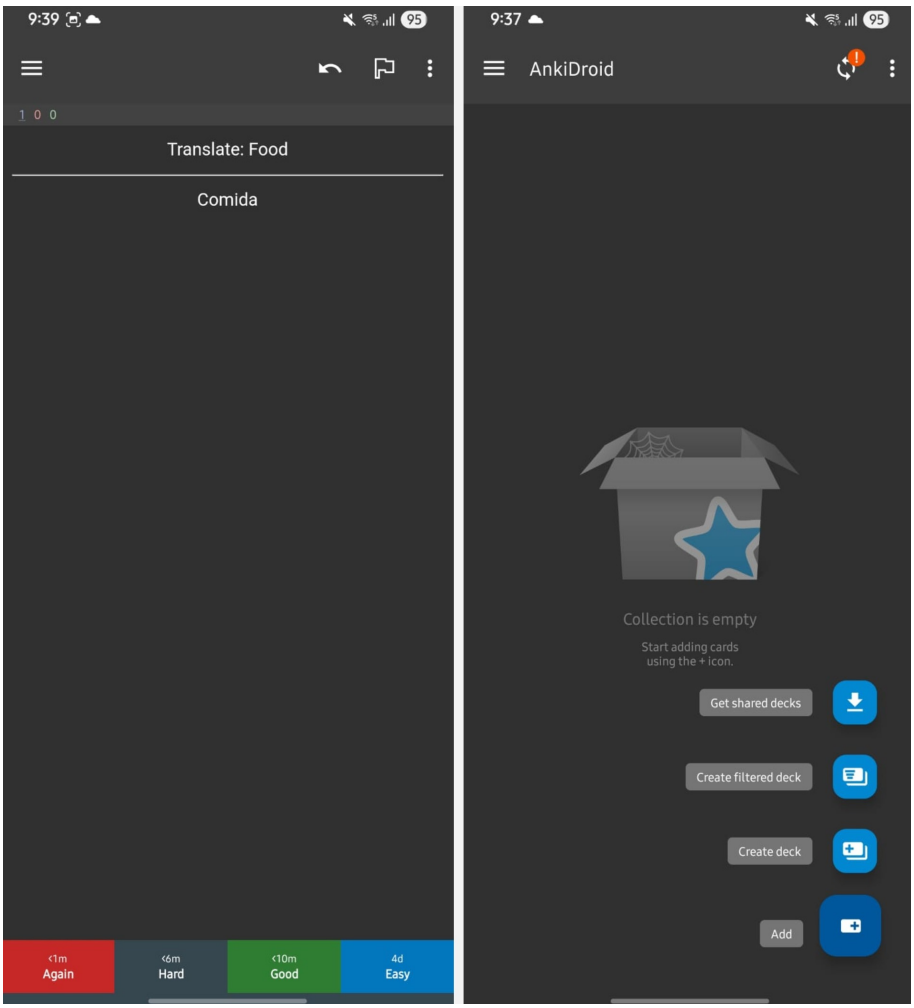
Figura 2 – Tela inicial do StudyTok



Fonte: (STUDYTOK, 2026).

Além disso, o aplicativo Anki, mostrado na Figura 3, é referência em repetição espaçada, oferecendo um sistema de *flashcards* com algoritmo que agenda a revisão de cada cartão para quando o usuário está mais propenso a esquecer a informação (ANKI, 2026). Apesar de ser o padrão de mercado nesse pilar específico, o Anki não possui algum recurso nativo de gestão de tempo, de gamificação ou de monitoramento de bem-estar, concentrando-se exclusivamente na otimização da memorização (ANKI, 2026).

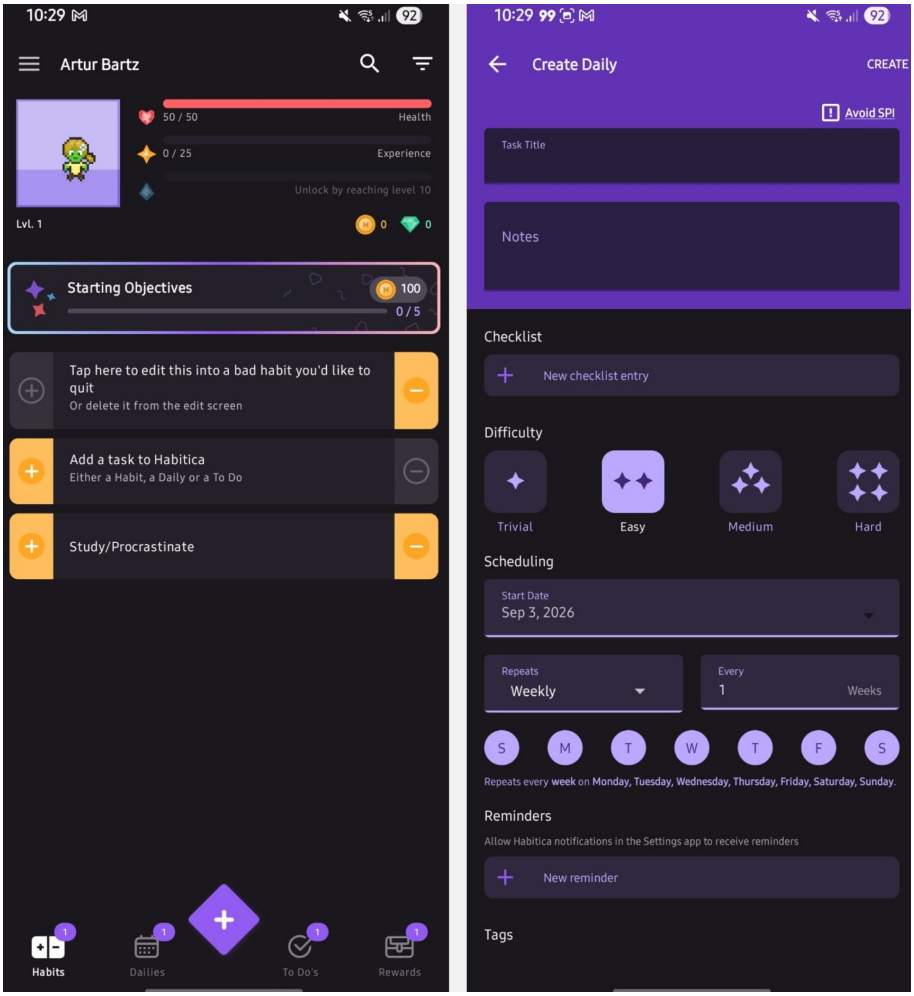
Figura 3 – Tela inicial do Anki (AnkiDroid)



Fonte: (ANKI, 2026).

Outro exemplo é o aplicativo Habitica, que adota a abordagem de ser uma plataforma de gamificação de hábitos e tarefas no formato de um jogo, na qual o usuário ganha experiência, ouro e itens ao concluir tarefas do cotidiano, além de poder participar de grupos e missões em equipe (HABITICA, 2026). Por ser uma ferramenta de produtividade genérica, e não voltada especificamente ao estudo, o Habitica não possui repetição espaçada, *flashcards*, cronômetro Pomodoro ou *check-ins* de humor. Sua gamificação se aplica a qualquer tipo de tarefa que o usuário cadastre, sem qualquer relação com técnicas de aprendizagem (HABITICA, 2026).

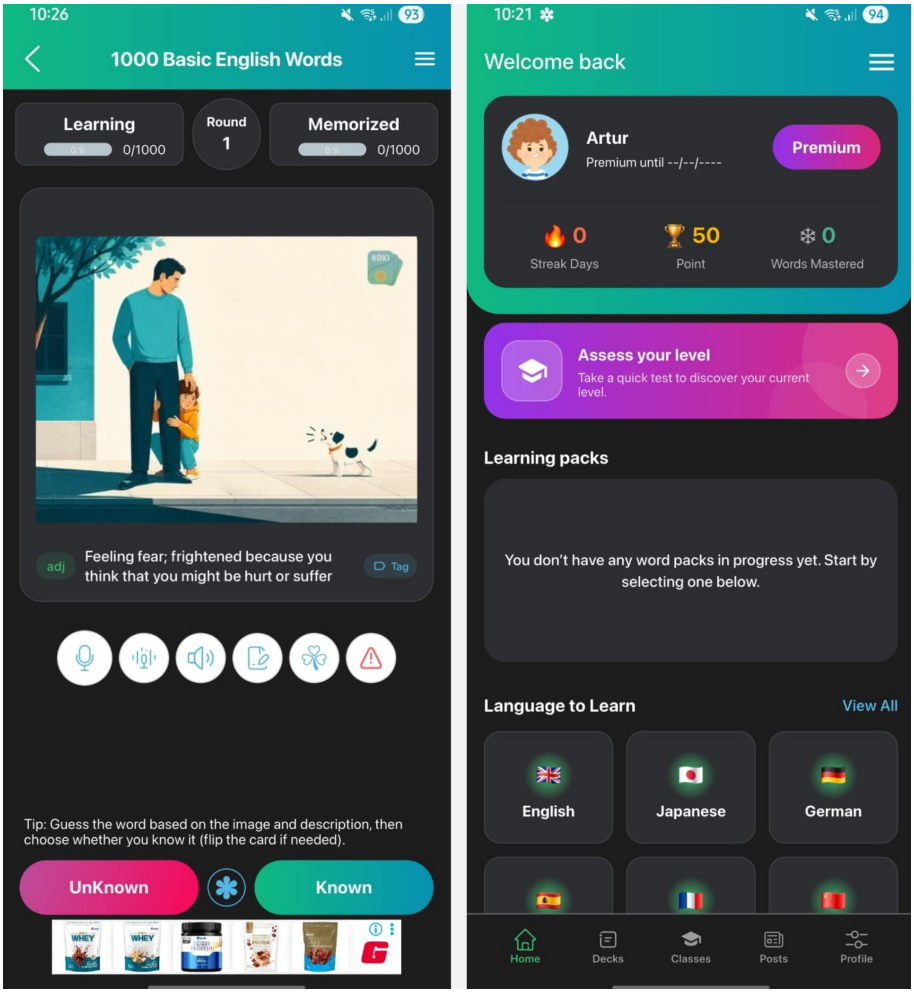
Figura 4 – Tela inicial do Habitica



Fonte: (HABITICA, 2026).

Por fim, o aplicativo Koki *FlashCards*, mostrado na Figura 5, é um aplicativo que aplica um algoritmo de repetição espaçada similar ao Anki, porém voltado à memorização de vocabulário de idiomas, com baralhos prontos para exames (KOKI, 2026). Diferentemente do Anki, o Koki incorpora elementos leves de gamificação, como sequência de dias de uso (*streak*) e pontuação, além de um plano de assinatura *premium* (KOKI, 2026). Ainda assim, por ser voltado exclusivamente à aprendizagem de vocabulário de idiomas, o Koki não possui cronômetro Pomodoro, gestão geral de tempo de estudo ou *check-ins* de bem-estar, e seus baralhos prontos não se aplicam ao conteúdo específico de disciplinas universitárias.

Figura 5 – Tela inicial do Koki *FlashCards*



Fonte: (KOKI, 2026).

Portanto, o Focus se diferencia dos concorrentes ao buscar uma integração completa de todos os valores mencionados anteriormente, em vez de se aprofundar isoladamente em um ou dois pilares. Enquanto o Aprovado se restringe ao registro e à estatística da gestão do tempo, o StudyTok se destaca na geração de *flashcards* por IA e gamificação, o Anki é o padrão de repetição espaçada, o Habitica é o padrão de gamificação de hábitos e o Koki combina repetição espaçada com gamificação leve voltada exclusivamente a vocabulário de idiomas, nenhum deles combina todos os propósitos com o controle de bem-estar do estudante. O Focus une esses valores a fim de trazer um produto centralizado e desfragmentado, evidenciando de forma mais robusta a lacuna de mercado que motivou este trabalho. O diferencial está em combinar a eficácia da repetição espaçada com uma ferramenta de cronômetro Pomodoro, elementos de gamificação e *check-ins* de humor, oferecendo uma solução mais adaptada às demandas do estudante universitário.

A Tabela 1 resume comparativamente os propósitos deste trabalho presentes em cada um dos cinco concorrentes analisados, evidenciando visualmente a lacuna de mercado descrita ao longo desta seção.

Tabela 1 – Comparativo entre o Focus e os concorrentes analisados

Aplicativo	**Gestão do tempo**	**SRS / *Flash-cards***	**Gamificação**	**Bem-estar**	**Conteúdo genérico universitário**
Aprovado	Sim	Não	Não	Não	Sim
StudyTok	Não	Sim	Sim	Não	Sim
Anki	Não	Sim	Não	Não	Sim
Habitica	Não	Não	Sim	Não	Sim
Koki *Flash-Cards*	Não	Sim	Sim	Não	Não
Focus	Sim	Sim	Sim	Sim	Sim

Fonte: autoria própria.

2.7 Heurísticas de Nielsen

A boa usabilidade de um sistema é construída a partir de um conjunto de regras reconhecidas no campo da engenharia de *software* como as heurísticas de Nielsen (NIELSEN, 1994). São dez princípios que servem como um guia para a implementação de interfaces digitais centradas nas necessidades do usuário. As heurísticas visam garantir que o sistema não apenas funcione, mas que seja acessível e fácil de usar.

Segundo essas diretrizes, o sistema deve: (i) sempre manter o usuário informado sobre o que está acontecendo, com *feedbacks*; (ii) usar linguagens conhecidas pelo usuário enquanto segue convenções já estabelecidas; (iii) oferecer oportunidades de cancelamento de operações indesejadas; (iv) os elementos do aplicativo devem se comportar de forma semelhante; (v) evitar erros antes que eles surgam; (vi) tornar objetos, ações e opções visíveis; (vii) incluir atalhos para usuários experientes; (viii) excluir componentes com informações irrelevantes; (ix) informar especificamente qual erro ocorreu, em sua mensagem; e (x) implementar uma documentação de auxílio.

Dessa forma, este trabalho se compromete a aplicar essas heurísticas como guia de *design*, buscando favorecer a usabilidade e a acessibilidade para diferentes públicos. Além disso, o projeto observa os princípios da Lei Geral de Proteção de Dados (LGPD) (BRASIL, 2018), orientando as decisões de tratamento das informações dos usuários; a Seção 4.2.3 detalha as medidas técnicas adotadas para essa conformidade.

2.8 Algoritmo de Repetição Espaçada (ARE)

A metodologia do algoritmo de repetição espaçada (ARE) foca na ideia da memorização de longo prazo, buscando otimizar o tempo de estudo ao confrontar a curva do esquecimento. Diante disso, o ARE calcula um intervalo de revisão para o usuário conforme a dificuldade percebida, buscando apresentar o conteúdo próximo ao momento em que estaria sendo esquecido, o que a literatura aponta como mais eficaz do que a repetição excessiva. A implementação dessa metodologia ganhou popularidade em aplicações móveis, pois a portabilidade dos *smartphones* reduz as limitações de tempo e local, permitindo que estudantes estudem em momentos e locais mais flexíveis. Nesse contexto, estudos como o de Kose e Mede (2018) investigaram os efeitos de uma aplicação de *flashcards* móveis com sistema de repetição espaçada sobre o desenvolvimento do vocabulário e a motivação dos estudantes, demonstrando que essa implementação resulta em aumentos mais altos de vocabulário e maior motivação.

Os algoritmos de repetição espaçada se classificam como uniforme ou dinâmico, este sendo mais eficiente já que calcula o tamanho ideal do intervalo da repetição con-

forme o *feedback*. O objetivo desses algoritmos, especialmente o espaçamento dinâmico, é capacitar os estudantes a dedicarem mais tempo aos itens que exigem mais sessões de revisão e menos tempo àqueles que já estão consolidados. Esse princípio deriva dos estudos de Wozniak e Gorzelanczyk (1994) sobre o espaçamento ótimo de repetições na prática do aprendizado, que fundamentam o algoritmo SuperMemo. Este trabalho utiliza uma variação do algoritmo dinâmico SuperMemo 2 (SM-2), em que os dois primeiros intervalos são fixos e os seguintes são calculados multiplicando o intervalo anterior pelo fator de facilidade (calculado com base na nota de facilidade fornecida pelo usuário).

2.9 Pomodoro *Timer*

O cronômetro Pomodoro é uma metodologia de gestão do tempo que estrutura o trabalho ou estudo em intervalos de foco, intercalados com pausas curtas e obrigatórias, com o objetivo de aumentar a produtividade e evitar a fadiga. Criada por Francesco Cirillo (CIRILLO, 2018), a técnica divide o tempo em intervalos de 25 minutos de foco, seguidos por 5 minutos de descanso. A cada quatro ciclos completos, há uma pausa de 30 minutos. Essa abordagem se alinha com o valor de gestão do tempo deste trabalho, proporcionando ao estudante uma metodologia eficaz para ajudar no planejamento e execução das tarefas.

O cronômetro Pomodoro gera o incentivo de descanso na hora ideal a fim de ajudar na manutenção da atenção. Ao garantir pausas frequentes, a técnica permite que a pessoa se recupere, evitando a sobrecarga e o declínio na retenção de informações. Dessa forma, a relevância dessa metodologia é crescente, com pesquisas que buscam avaliar sua aplicabilidade em campos de estudos. Um exemplo disso é a revisão de escopo realizada por Ogut (2025), que teve como objetivo avaliar a relevância da técnica Pomodoro na retenção de lições de anatomia demonstrando a busca por evidências da eficácia da técnica em ambientes acadêmicos. O estudo aponta que mesmo intervalos estruturados como 24 minutos de trabalho seguidos por 6 minutos de descanso ou 12 minutos de trabalho seguidos por 3 minutos de descanso levaram a resultados positivos em provas. Dessa forma, o aplicativo implementa a funcionalidade de um cronômetro Pomodoro com ciclos configuráveis, permitindo que o usuário ajuste os tempos de foco e pausa de acordo com sua preferência.

2.10 Aprendizagem Autorregulada (*Check-ins* de Humor)

A aprendizagem autorregulada (*Self-Regulated Learning* - SRL) descreve o processo pelo qual os estudantes monitoram e controlam seu aprendizado. A capacidade de autorregulação permite que o estudante ajuste seu comportamento e planejamento de es-

tudo em resposta às demandas e ao seu estado mental. Essa importância se tornou ainda mais evidente em contextos de estudo autodidata, como o ensino remoto causado pela COVID-19, onde o suporte do professor foi limitado, exigindo maior independência do aluno.

O estado psicológico do estudante impacta diretamente a absorção de informação e o engajamento com a tarefa. A metodologia SRL diz que, se o estudante está ciente de que seu humor está baixo ou seu estresse alto, ele pode usar estratégias de autorregulação para mitigar esse efeito negativo, como ajustar a dificuldade da tarefa, alterar o foco de estudo ou tirar uma pausa. O estudo de Ebbes et al. (2026), ao entrevistar professores sobre o ensino remoto, destacou que um dos principais desafios enfrentados pelos educadores era justamente o suporte ao bem-estar emocional dos alunos.

Dessa forma, a integração de *check-ins* de bem-estar e a análise dos gráficos gerados transformam o aplicativo em um agente de suporte à SRL. Ao coletar dados sobre o estado emocional, o sistema pode oferecer uma oportunidade de análise desses índices, permitindo o surgimento de ideias de intervenção que promovem a saúde mental e otimizam a produtividade. Essa funcionalidade metodológica atende diretamente à necessidade, identificada em pesquisas como a de Ebbes et al. (2026), de monitorar o engajamento e o bem-estar emocional do aluno em um ambiente de estudo individual, atuando como um professor que oferece esse suporte.

2.11 Autossabotagem e Adesão ao Uso do Aplicativo

Além de organizar a rotina de estudos, o aplicativo deste trabalho precisa lidar com o problema comportamental de abandono do aplicativo. Uma revisão recente, que analisou dezoito estudos e mais de quinhentos mil participantes, identificou que aproximadamente 70% dos usuários interrompem o uso de aplicativos de comportamento e saúde mental dentro dos primeiros 100 dias, com desistências acentuadas já nos estágios iniciais (KIDMAN et al., 2024). Esse comportamento está ligado ao objetivo de negócio deste trabalho de minimizar a taxa de abandono, e motiva a implementação de funcionalidades de *design* que combatem formas de autossabotagem do usuário.

A primeira dessas implementações combate o padrão do indivíduo desistir do seu compromisso de estudo ao falhar apenas um dia. Esse padrão se manifesta em sistemas gamificados de sequências (*streaks*) de hábitos e estudo, ou seja, ao falhar um único dia, o usuário visualiza sua sequência zerada como prova de fracasso e desiste de continuar usando o aplicativo. Para mitigar isso, o aplicativo implementa uma funcionalidade de congelamento de *streak*, que perdoa a perda de um único dia por semana sem zerar a sequência do usuário, permitindo que uma falha pontual seja tratada como uma exceção

esperada, e não como o fim do esforço.

A segunda funcionalidade reduz a sobrecarga de decisão no momento em que o usuário retoma os estudos após um período de ausência. A literatura sobre sobrecarga de escolha (*choice overload*) demonstra que, quando confrontado com um número excessivo de opções ou itens pendentes, o indivíduo tende a adiar ou evitar a decisão de agir, em vez de se sentir motivado por ela (CHERNEV; BÖCKENHOLT; GOODMAN, 2015). Aplicado ao contexto deste trabalho, quando um usuário se ausenta por alguns dias e retorna ao aplicativo, este encontra dezenas de revisões de *flashcards* acumuladas e tende a se sentir sobrecarregado, adiando ou evitando a sessão de estudo. Por isso, em vez de exibir de uma só vez todas as revisões pendentes, o aplicativo limita a lista exibida por sessão a um lote reduzido, revelando o restante posteriormente, à medida que o usuário avança.

Finalmente, a terceira funcionalidade estende a ideia de *check-in* de humor de um simples registro para uma intervenção ativa. Quando o sistema identifica um padrão recorrente de *check-ins* negativos, o aplicativo emite um alerta com uma sugestão de baixo esforço ou pausa em vez de apenas servir como dados para os gráficos de progresso. Portanto, ao tornar visível o próprio estado emocional, o usuário ganha a oportunidade de ajustar seu comportamento antes que o desgaste o leve a abandonar os estudos por completo (LIU; LU; YAN, 2025).

3 Metodologia

Neste capítulo é apresentado o planejamento metodológico que orientou o desenvolvimento do projeto. São descritos o processo de pesquisa, as metodologias ágeis de desenvolvimento escolhidas e os artefatos de levantamento de requisitos.

3.1 Processo Metodológico

Conforme o proposto pela Universidade de Brasília, o trabalho de conclusão de curso é estruturado em duas etapas. A primeira etapa dedica-se à elaboração de uma proposta de projeto para desenvolvimento e à construção e documentação do seu referencial teórico. De outra maneira, a segunda etapa concentra-se na execução e implementação prática do projeto que foi definido durante a fase anterior.

3.1.1 TCC1 e TCC2

A organização e o detalhamento das etapas que compõem a primeira e a segunda etapa do trabalho de conclusão de curso estão apresentados abaixo.

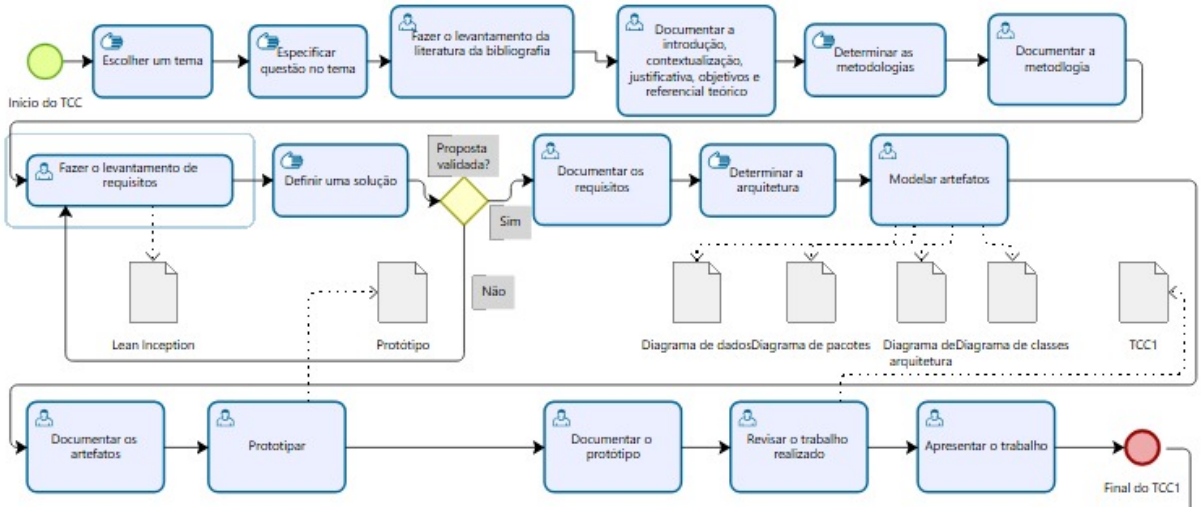
- (TCC1) Semana do dia 15 de março de 2026: Escolher um tema.
- (TCC1) Semana do dia 22 de março de 2026: Especificar a questão no tema.
- (TCC1) Semana do dia 29 de março de 2026: Determinar uma metodologia.
- (TCC1) Semana do dia 05 de abril de 2026: Fazer levantamento da literatura.
- (TCC1) Semana do dia 12 de abril de 2026: Fazer levantamento da literatura.
- (TCC1) Semana do dia 19 de abril de 2026: Levantamento de requisitos.
- (TCC1) Semana do dia 26 de abril de 2026: Levantamento de requisitos.
- (TCC1) Semana do dia 03 de maio de 2026: Definir uma solução.
- (TCC1) Semana do dia 10 de maio de 2026: Definir uma solução.
- (TCC1) Semana do dia 17 de maio de 2026: Determinar a arquitetura.
- (TCC1) Semana do dia 24 de maio de 2026: Modelar artefatos.
- (TCC1) Semana do dia 31 de maio de 2026: Modelar artefatos.

- (TCC1) Semana do dia 07 de junho de 2026: Modelar artefatos.
- (TCC1) Semana do dia 14 de junho de 2026: Prototipar.
- (TCC1) Semana do dia 21 de junho de 2026: Prototipar.
- (TCC1) Semana do dia 28 de junho de 2026: Prototipar.
- (TCC1) Semana do dia 05 de julho de 2026: Revisar o trabalho realizado.
- (TCC1) Semana do dia 12 de julho de 2026: Apresentar o trabalho.
- (TCC2) Semana do dia 09 de agosto de 2026: Corrigir levantamentos da banca.
- (TCC2) Semana do dia 09 de agosto de 2026: Implementar o CRUD de perfil.
- (TCC2) Semana do dia 16 de agosto de 2026: Implementar o cadastro e autenticação.
- (TCC2) Semana do dia 23 de agosto de 2026: Implementar a tela *home* e o cronômetro Pomodoro.
- (TCC2) Semana do dia 30 de agosto de 2026: Implementar o CRUD de *flashcards* e auto-avaliação.
- (TCC2) Semana do dia 06 de setembro de 2026: Implementar a configuração de durações.
- (TCC2) Semana do dia 13 de setembro de 2026: Implementar *Check-in* de bem-estar e Revisão SRS.
- (TCC2) Semana do dia 20 de setembro de 2026: Implementar barra de progresso visual e lembrete de revisão SRS.
- (TCC2) Semana do dia 27 de setembro de 2026: Implementar estatísticas de foco e *deck* por disciplina.
- (TCC2) Semana do dia 04 de outubro de 2026: Implementar XP, níveis e gráficos.
- (TCC2) Semana do dia 11 de outubro de 2026: Implementar o *onboarding*.
- (TCC2) Semana do dia 18 de outubro de 2026: Implementar o *streak* diário.
- (TCC2) Semana do dia 25 de outubro de 2026: Validar o MVP com o orientador.
- (TCC2) Semana do dia 01 de novembro de 2026: Corrigir apontamentos do orientador.
- (TCC2) Semana do dia 08 de novembro de 2026: Documentar o desenvolvimento e a solução final.

- (TCC2) Semana do dia 15 de novembro de 2026: Documentar os resultados.
- (TCC2) Semana do dia 22 de novembro de 2026: Revisar o trabalho realizado.
- (TCC2) Semana do dia 29 de novembro de 2026: Prearar apresentação final.
- (TCC2) Semana do dia 06 de dezembro de 2026: Apresentar o trabalho final.

A seguir, está representado o diagrama de fluxo de atividades BPMN desta primeira etapa.

Figura 6 – Raia do TCC1 do BPMN do Processo Metodológico.

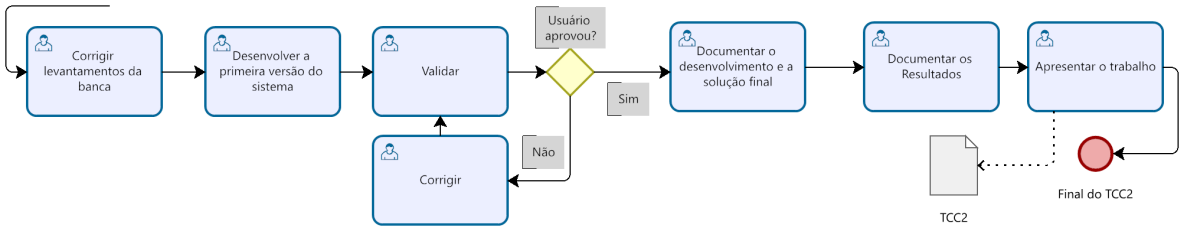


Fonte: autoria própria.

3.1.2 TCC2

A seguir, está representado o diagrama de fluxo de atividades BPMN da segunda etapa do trabalho de conclusão de curso, realizada no semestre posterior à aprovação da proposta.

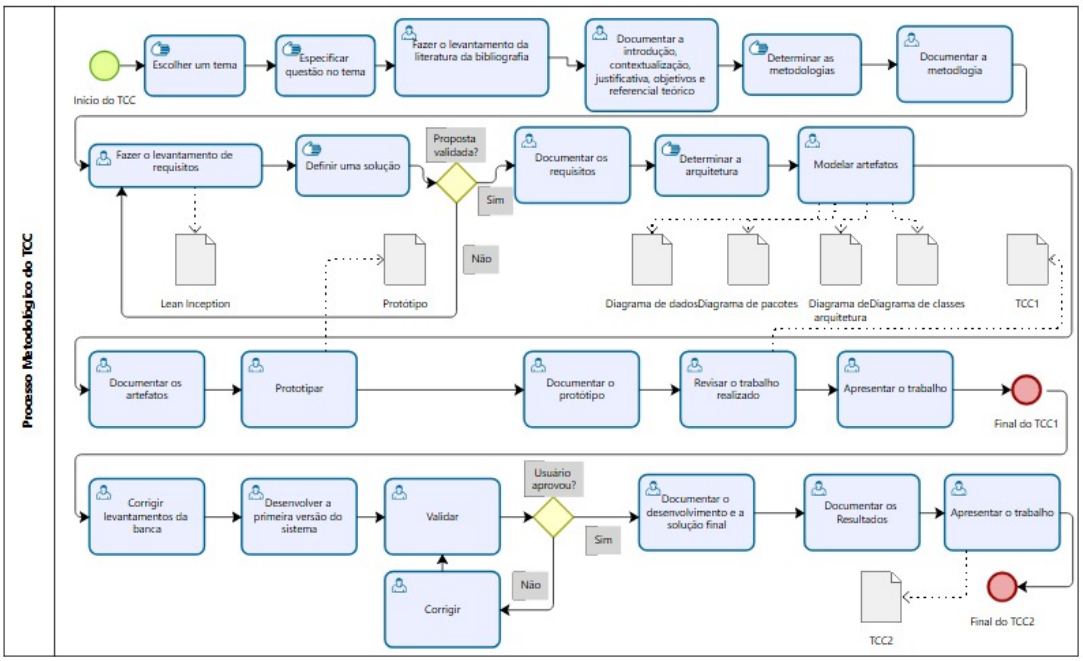
Figura 7 – Raia do TCC2 do BPMN do Processo Metodológico.



Fonte: autoria própria.

Portanto, podemos representar todo o fluxo de atividades de ambas as etapas no seguinte diagrama BPMN.

Figura 8 – BPMN do Processo Metodológico



Fonte: autoria própria.

3.2 Metodologia de Desenvolvimento

A escolha da metodologia de desenvolvimento de *software* é um fator que define a forma como o projeto foi planejado, executado e gerenciado, afetando na capacidade de entrega e na adaptação à mudança dos requisitos. Antigamente, o desenvolvimento era feito com modelos preditivos, como o modelo cascata, que exige o levantamento completo de requisitos antes do desenvolvimento em si. Contudo, o mercado de *software* mudou para abordagens mais ágeis. Essa transição reflete a necessidade de lidar com a complexidade e a volatilidade do ambiente de negócios moderno, onde as necessidades dos usuários evoluem rapidamente.

A metodologia ágil, representada por *frameworks* como *Scrum*, Kanban e *Extreme Programming* (XP), é a mais adequada para o desenvolvimento de soluções centradas no usuário, como o Focus, aplicativo desenvolvido neste trabalho. Em contraste com o modelo cascata, o ágil valoriza a entrega contínua de *software* e a adaptação a requisitos novos. Em um estudo realizado em uma instituição de ensino superior, Santos, Filipe e Cunha (2024) analisaram a transição do cascata para o *Scrum* em um curso de gerenciamento de projetos e demonstraram que, seguindo o *Scrum*, os estudantes se tornaram mais capazes de desenvolver *software* em equipes, com maior foco nos clientes, e adquiriram mais conhecimento em áreas fundamentais do desenvolvimento de *software*.

3.2.1 Metodologia Ágil

Dessa forma, a escolha por uma metodologia de desenvolvimento ágil é uma estratégia que evita riscos e otimiza o ciclo de vida do projeto. Ao aderir a *frameworks* ágeis, este trabalho flexibiliza o plano de desenvolvimento, permitindo ajustes rápidos caso surjam a necessidade de alterar ou refinar requisitos ao longo do processo. Além disso, as entregas curtas e frequentes proporcionam um processo mais produtivo, onde o progresso é continuamente visível.

Contudo, o projeto adotou as metodologias ágeis para garantir a flexibilidade e aumentar a produtividade. Essa abordagem metodológica, conforme validada pela experiência educacional descrita por Santos, Filipe e Cunha (2024), é ideal para desenvolver *software*. Focar nas necessidades do usuário e evitar os riscos de desvios de escopo ou de incompatibilidade do produto final resultam em uma implementação mais eficiente e centrada no cliente.

3.2.2 *Extreme Programming* (XP)

O *Extreme Programming* (XP) é uma metodologia ágil focada na entrega contínua em ambientes onde os requisitos mudam frequentemente. A metodologia é definida por uma série de práticas agrupadas em quatro áreas centrais: planejamento, *design*, codificação e teste. A ideia central do XP está no valor, na simplicidade, na comunicação constante e na motivação para refatorar o código. Essa importância na adaptabilidade e na qualidade do código faz do XP uma boa escolha para projetos que buscam resultados que satisfaçam o usuário.

A metodologia XP é eficaz em projetos que precisam de integração rápida, como o desenvolvimento de uma plataforma que unifica várias ideias. Os processos do XP incluem o desenvolvimento guiado por testes e a programação em par. Essas práticas visam mitigar erros, garantir a qualidade e transferir conhecimento de forma eficiente dentro da equipe. Sasmito et al. (2025) demonstraram a implementação do método XP no desenvolvimento de um *website* para gerenciamento de pedidos de transporte de colheitas agrícolas, confirmando que o XP pode ser aplicado com sucesso para a construção de sistemas que gerenciam dados complexos e exigem alta funcionalidade.

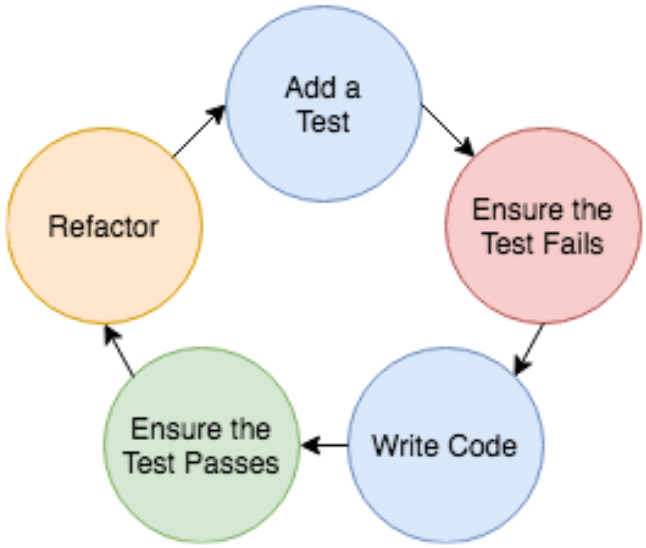
Para o contexto deste trabalho, o XP oferece um modelo metodológico de alto desempenho, complementando o *framework Scrum* adaptado e o quadro Kanban. O XP pode ser utilizado para definir as práticas de engenharia dentro de cada *Sprint* do *Scrum*, garantindo que a implementação dos requisitos seja feita com qualidade e erro técnico mínimo. Entretanto, devido ao escopo do projeto, não é viável adotar todas as práticas do método, sendo priorizado apenas os testes TDD (*Test-Driven Development*) e a

refatoração.

3.2.3 *Test-Driven Development*

O *Test-Driven Development* (TDD), ou Desenvolvimento Orientado por Testes, é uma prática essencial das metodologias ágeis que inverte a ordem normal da codificação. Em vez de escrever o código e testá-lo, o TDD exige que os desenvolvedores escrevam o teste antes de escrever o seu código. O processo segue o ciclo: o teste falha, o desenvolvedor escreve o código necessário para fazer o teste passar e, em seguida, refatora o código. Essa prática assegura que cada parte do *software* atenda a um requisito específico e seja verificável, promovendo uma base de código com alta taxa de testes.

Figura 9 – Ciclo de desenvolvimento TDD



Fonte: (TESTDRIVEN, 2025).

A contribuição do TDD para o *software* é a qualidade do produto final e a geração de um código modular para outros desenvolvedores. Ao forçar o desenvolvedor a pensar nas especificações antes de codificar, o TDD resulta em um *design* mais simples e modular, pois o código se torna mais fácil de testar. Essa modularidade simplifica a manutenção e a inclusão de novas funcionalidades no futuro.

A metodologia TDD é fundamental para o sucesso do Focus, especialmente na implementação de componentes como o algoritmo de repetição espaçada (SRS). A precisão dos cálculos do SRS e a integração do cronômetro Pomodoro com a gamificação não podem depender de testes manuais. O TDD busca assegurar que cada regra de negócio do algoritmo seja coberta por um teste, diminuindo a probabilidade de erros de cálculo não detectados. Dessa forma, a adesão ao TDD, conforme discutido por Cassieri et al.

(2025), tende a contribuir para a qualidade do código e para a verificação de que as funcionalidades do sistema operam de acordo com o referencial teórico e os requisitos definidos.

3.2.4 *Scrum*

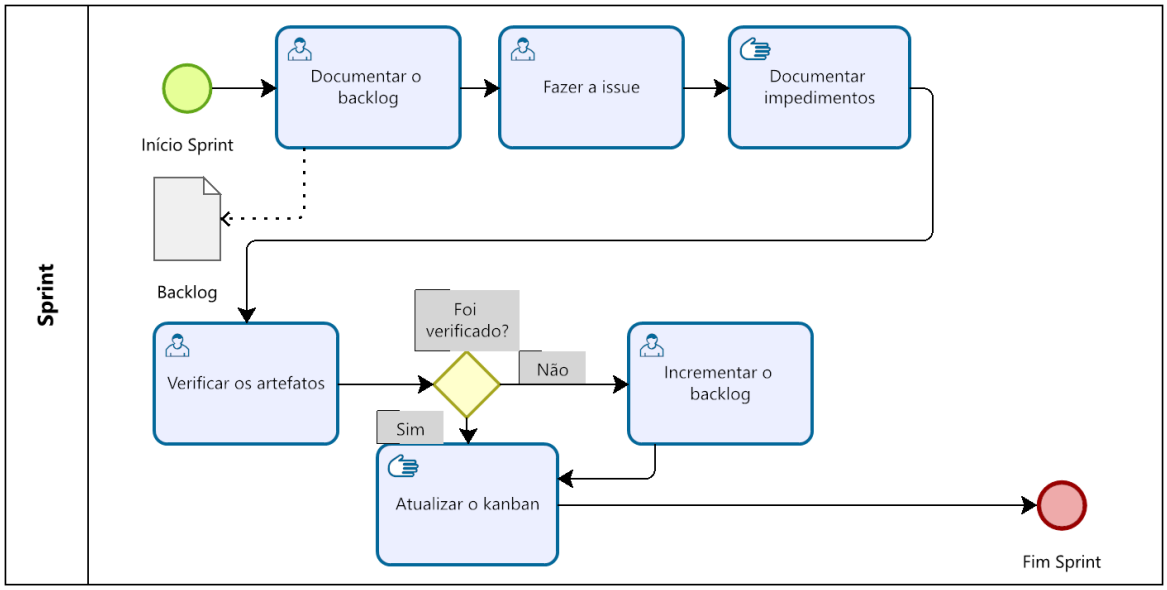
O *Scrum* é um *framework* ágil e muito utilizado que estabelece uma estrutura iterativa e incremental para o gerenciamento de projetos, ideal para o desenvolvimento de *software*. O *Scrum* é composto por ciclos repetitivos de trabalho chamados *Sprints*, com duração de uma a quatro semanas, normalmente. Sua principal função é facilitar a entrega rápida de valor, permitindo que o produto seja refinado com base no *feedback* do cliente e nos novos requisitos a cada iteração do ciclo. O *framework* define papéis e eventos que mantém a equipe alinhada com os objetivos do projeto.

A metodologia *Scrum* é fundamental para este trabalho, pois este exige a integração de múltiplas funcionalidades que precisam ser desenvolvidas em fases paralelas. O uso do *Scrum* permite que cada *Sprint* se concentre na entrega de apenas um incremento, mitigando o risco de desvios de escopo. Ibarra-Torres, Urbieta e Medina-Medina (2025) destacam em seus estudos que a aplicação do *Scrum* pode ser uma ferramenta eficaz para a transferência de conhecimento entre desenvolvedores, especialmente através dos eventos como o *Daily Scrum* (reuniões curtas e diárias) e as sessões de programação em pares.

Dessa forma, o *Scrum* foi utilizado para demonstrar o incremento do *software* ao orientador e coletar o *feedback* dele, garantindo a melhoria contínua do processo de desenvolvimento. Apesar do trabalho ser realizado por uma pessoa, a documentação dos processos e impedimentos guardam conhecimentos importantes para manter o cronograma em dia. A adoção do *Scrum* adaptado como sugerido por Ibarra-Torres, Urbieta e Medina-Medina (2025), fornece a estrutura ideal para gerenciar a complexidade técnica do aplicativo, garantindo que o resultado final seja de alta qualidade e alinhado com o referencial teórico do trabalho.

A seguir está representado o fluxo de atividades da *sprint* deste trabalho.

Figura 10 – BPMN *Sprint*



Fonte: autoria própria.

3.2.5 Kanban

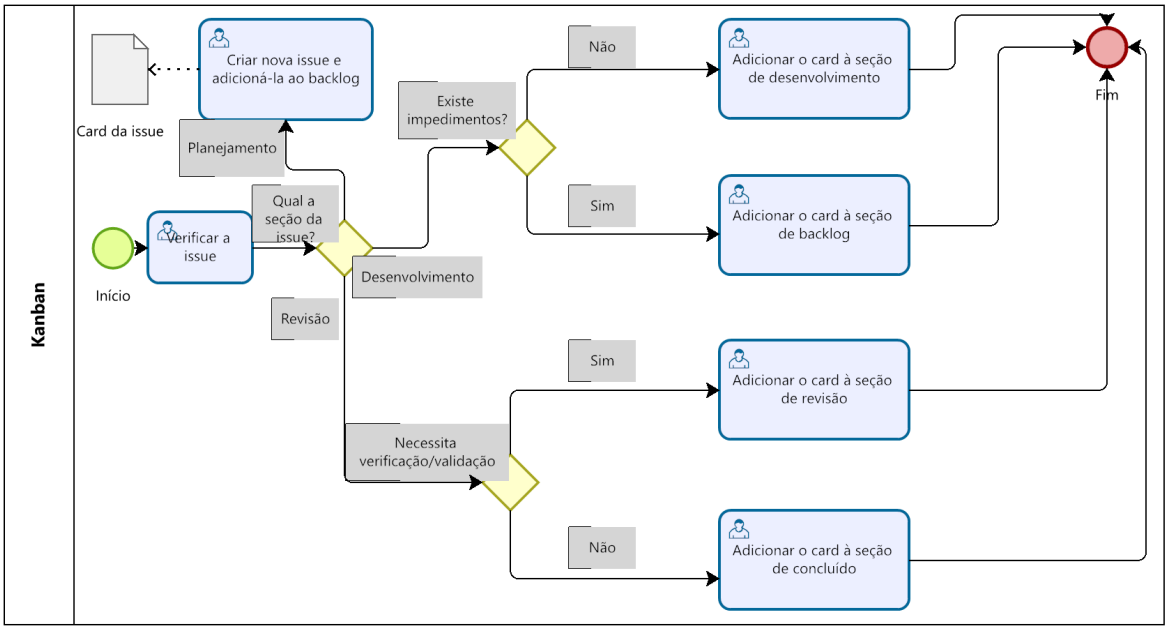
O Kanban é uma metodologia ágil que foca na gestão visual do fluxo de trabalho e na limitação do trabalho em progresso. Originária da indústria automobilística japonesa, o Kanban foi adaptado para o desenvolvimento de *software*, onde o trabalho só é iniciado quando há disponibilidade. O valor central do Kanban é o seu quadro, que divide o trabalho em colunas representando os estágios do processo. Ao visualizar o fluxo, a equipe pode identificar problemas, otimizar o tempo e focar na entrega contínua.

Ao limitar o trabalho em progresso, o Kanban impede que a equipe inicie várias tarefas ao mesmo tempo, o que leva à perda de foco e a longos tempos de ciclo. Em um estudo sobre o impacto do Kanban na Aprendizagem Baseada em Problemas (ABP), Loli et al. (2025) concluíram que a aplicação do método Kanban em estudantes universitários demonstrou efeitos positivos, destacando o potencial da ferramenta para organizar e visualizar o desenvolvimento de tarefas complexas em um contexto acadêmico.

Para o desenvolvimento deste trabalho, o Kanban foi empregado como um sistema visual de acompanhamento, complementando o *Scrum* e atuando como o principal mecanismo de gestão do fluxo de trabalho. Dessa forma, o quadro Kanban foi usado para rastrear o progresso das tarefas de desenvolvimento do aplicativo. A utilização do Kanban permitirá ao desenvolvedor e ao orientador inspecionar o andamento do projeto de forma transparente, facilitando a identificação de bloqueios ou atrasos e garantindo que a capacidade de trabalho esteja sempre focada na próxima funcionalidade de maior valor. A seguir está representado o fluxo de atividades da atualização do quadro Kanban do

projeto.

Figura 11 – BPMN Kanban



Fonte: autoria própria.

3.2.6 *Lean Inception*

O *Lean Inception* é uma abordagem estruturada, criada por Paulo Caroli, que se destina a alinhar o desenvolvedor, os *stakeholders* e o cliente em torno da visão de um Produto Mínimo Viável (MVP) em um período de tempo. Essa metodologia combina os princípios do *Design Thinking*, *Lean Startup* e Metodologias Ágeis. O principal objetivo é garantir que o desenvolvimento de um *software* comece com uma compreensão clara do problema a ser resolvido e do valor que a solução deve entregar. Em projetos ágeis, onde os requisitos podem evoluir, o *Lean Inception* atua como uma fase crucial de definição dos requisitos, a fim de evitar o desperdício de recursos na implementação de funcionalidades desnecessárias.

Normalmente as metodologias de desenvolvimento não incluem os usuários finais na participação das etapas iniciais de definição do projeto. Em vez disso, a visão é definida por clientes relacionados apenas ao negócio, o que pode levar a um produto que não atenda às necessidades reais do usuário final. É neste ponto que o *Lean Inception* se destaca, fornecendo uma série de atividades estruturadas que buscam integrar a perspectiva do usuário desde o início, além de incluir as visões dos desenvolvedores e dos *stakeholders*. Conforme apontado por Ferreira et al. (2024), o *Lean Inception* representa uma metodologia que combina o *Design Thinking* e o *Lean Startup* para apoiar a definição do problema e a produção eficaz, focando nas necessidades do usuário.

Portanto, a adoção do *framework Lean Inception* contribui para a fase de planejamento, apoiando a construção de uma proposta de valor mais clara e orientada às necessidades levantadas junto aos participantes do processo. O *Lean Inception* serve como um guia para transformar os requisitos levantados em artefatos para o desenvolvimento. Ao passar por todas as etapas, o projeto definiu o menor conjunto de funcionalidades capaz de entregar o maior valor percebido, reduzindo o risco de construir um produto distante das demandas identificadas. A estrutura do *Lean Inception* favorece uma definição mais clara do problema e um maior envolvimento dos participantes, conforme investigado por Ferreira et al. (2024). Para este trabalho, o *Lean Inception* foi conduzido por duas pessoas (representando usuários finais) e o autor (representando negócio e desenvolvimento) em um período de 5 dias – uma amostra reduzida, cujas hipóteses levantadas ainda carecem de validação com uma base maior de usuários. Ao final do processo, houve a revisão do orientador sobre os artefatos gerados. A seguir, é explicada cada etapa do *framework*, assim como os artefatos gerados.

Kick-off

O *Kick-off* é a etapa inicial do *Lean Inception*, atuando como o ponto de partida para alinhar todos os participantes em relação ao propósito e às regras. De acordo com Caroli (2017), o principal objetivo deste momento é garantir que a equipe e os *stakeholders* compartilhem uma compreensão correta sobre o que será feito e sobre o porquê de o projeto existir. Essa seção introdutória é onde são apresentados os participantes, o contexto do projeto, e a agenda das atividades do *framework*. Ao finalizar o *Kick-off*, o projeto estabelece o tom ágil e colaborativo necessário para a construção do MVP, minimizando o risco de que o produto final não reflita os objetivos e as necessidades práticas do usuário, conforme orienta a metodologia (CAROLI, 2017).

Visão de Produto

A Visão de Produto é o primeiro artefato de alto nível que surge do *Lean Inception* e serve como a ideia estratégica do projeto, englobando o propósito do aplicativo e o efeito que se deseja alcançar junto ao usuário. Segundo Caroli (2017), uma visão de produto eficaz deve ser concisa, inspiradora e orientada ao valor, atuando como um guia para a tomada de decisões futuras sobre o que incluir ou excluir do escopo do MVP. Ela responde à seguinte pergunta: "Qual é o produto que vai ser construído e por que ele importa?".

Para estruturar essa visão, o *Lean Inception* utiliza o formato de *template* "Visão de Produto"de Geoffrey Moore, que preenche as seguintes perguntas: público-alvo, problema, nome, categoria do produto, principal benefício, concorrentes e diferencial primário. As respostas dessas perguntas garantem que o time de desenvolvimento compreenda o valor de negócio antes de se aprofundar na lista de funcionalidades. A seguir estão documentadas as respostas da visão de produto.

- **Público-alvo:** Estudantes universitários.
- **Problema:** Inexistência de aplicativos gratuitos que integram cronômetro Pomodoro, *flashcards* com repetição espaçada, gamificação e *check-ins* de humor.
- **Nome:** Focus.
- **Categoria:** Aplicativo *mobile*.
- **Benefício:** Aprimorar a rotina de estudos do usuário.
- **Concorrentes:** Aprovado, StudyTok, Anki, Habitica e Koki *FlashCards*.
- **Diferencial:** Integra as diferentes ideias em um único aplicativo.

É - Não é - Faz - Não faz

A matriz "É – Não É – Faz – Não Faz"é a etapa que delimita o escopo do MVP. Esta matriz força a equipe e os *stakeholders* a criarem um consenso sobre o que o produto será e o que ele fará em sua primeira versão. Segundo Caroli (2017), o uso dessa matriz é uma atividade de priorização e alinhamento, pois ao declarar o que o produto não é e o que ele não faz, a equipe está definindo limites que protegem o projeto contra o desvio de escopo (*scope creep*).

A matriz é dividida em quatro quadrantes que explicam a identidade e o comportamento do produto. A coluna "É"define a categoria do produto, enquanto a coluna "Não É"estabelece limites sobre o que o aplicativo não seria. De forma similar, a coluna "Faz"lista as ações principais do produto, já a coluna "Não Faz"lista funcionalidades que poderiam ser esperadas, mas não estão no escopo. A seguir está listada essa matriz para o presente trabalho.

- **É:** Aplicativo *mobile*, gratuito e *open source*.
- **Não é:** Um banco de questões ou conteúdos prontos de disciplinas.
- **Faz:** Integra o cronômetro Pomodoro, *flashcards* com repetição espaçada, gamificação e *check-ins* de humor para auxiliar na rotina de estudos dos usuários.
- **Não faz:** Não corrige questões, não possui uma base de dados de questões ou conteúdos, não cria atividades, não gera conteúdo e não utiliza inteligência artificial.

Objetivos do Negócio

Para este trabalho foram acordados os seguintes objetivos de negócio: eficácia da integração das metodologias, minimizar a taxa de abandono, fornecer uma interface intuitiva, fornecer *check-ins* de saúde e seção de progresso.

Eficácia da integração das metodologias

Este objetivo foi atingido ao implementar o uso da repetição espaçada com os ciclos de foco do cronômetro Pomodoro, garantindo que o estudante revise o material correto no momento ideal.

Minimizar a taxa de abandono

Este objetivo foi alcançado por meio dos elementos de gamificação e de mecanismos específicos de combate à autossabotagem (Seção 2.11): o congelamento semanal de *streak*, a limitação do lote de revisões pendentes exibidas por sessão – complementada pelo treino livre para os usuários que desejam ir além dela – e os alertas de intervenção emitidos a partir de *check-ins* de bem-estar recorrentemente negativos. Ao utilizar barras de progresso e pontos, o aplicativo transforma o esforço em uma experiência progressiva.

Fornecer uma interface intuitiva

A interface é minimalista, tornando as complexidades do SRS e do Pomodoro transparentes ao usuário. O foco está em simplificar a navegação e a interação, permitindo que o usuário dedique suas energias aos estudos, e não à compreensão de como usar o aplicativo.

Fornecer *check-ins* de saúde

A funcionalidade permite que o aplicativo monitore o estado psicológico do usuário diariamente. Essa informação é vital para o processo de autorregulação e análise dos gráficos gerados.

Seção de progresso

Essa seção mostra o progresso e as estatísticas do usuário de forma intuitiva.

Proto-Personas

A atividade de definição de personas dentro do *Lean Inception* é um exercício focado no usuário final. Uma persona é um usuário fictício que representa um conjunto de usuários e que, em sua formulação mais robusta, é construída a partir de pesquisas extensivas com usuários reais. Como a base empírica deste trabalho está restrita às visões da própria equipe do *Lean Inception* (duas pessoas representando usuários finais e o autor, representando negócio e desenvolvimento), sem uma etapa de pesquisa de campo mais ampla com o público-alvo, as representações a seguir são tratadas como *proto-personas*: hipóteses de perfil de usuário baseadas no conhecimento e nas suposições da equipe, que deverão ser validadas ou refinadas com usuários reais em etapas futuras do projeto. Segundo Caroli (2017), ainda que nesse formato inicial, o exercício ajuda a transformar necessidades abstratas do usuário final em características, metas e dores concretas, apoiando decisões de *design* e de funcionalidade mais centradas no usuário. A seguir estão representadas as três proto-personas deste trabalho.

Primeira Proto-Persona

- Perfil:
  - Nome: Icarus.
  - Idade: 20 anos.
  - Profissão: Estudante universitário.
- Objetivos:
  - Minimizar a taxa de abandono.
- Frustrações:
  - Não sabe por onde começar;
  - Sofre com a procrastinação e o *burnout*;
  - Iniciante na vida acadêmica.
- Necessidades:
  - Alta necessidade de estrutura e motivação.

Segunda Proto-Persona

- Perfil:
  - Nome: Hazel.
  - Idade: 27 anos.
  - Profissão: Estudante de pós-graduação.
- Objetivos:
  - Aprender métodos para tornar suas seções de estudos mais eficazes;
- Frustrações:
  - Tempo de estudo muito limitado;
  - Falta de tempo para gerenciar o cronograma;
  - Dificuldade em reter conteúdo após longas pausas;
  - Muito ocupada;
- Necessidades:
  - Eficácia da Integração das Metodologias.

Terceira Proto-Persona

- Perfil:
  - Nome: Arabella.
  - Idade: 25 anos.
  - Profissão: Estudante universitária.
- Objetivos:
  - Poder medir sua ansiedade e estresse durante os estudos.
- Frustrações:
  - Alto volume de conteúdo para revisar e consolidar;
  - Dificuldade em otimizar as revisões;
  - Estresse e fadiga mental pelo ritmo.
- Necessidades:
  - Receber e analisar seus *Check-ins* de Saúde e usar a Interface Intuitiva.

Jornadas de Usuário

A atividade de mapeamento das Jornadas de Usuário do *Lean Inception* consiste em visualizar a sequência de passos, interações e emoções que cada Persona experimenta ao tentar atingir seus objetivos principais utilizando o produto. A seguir estão representadas as três jornadas.

Tabela 2 – Jornada de Icarus

Etapa da Jornada	**Ação da Persona (Icarus)**	**Sentimento**	**Ponto de Dor / Desafio**	**Ação do Aplicativo (Funcionalidade)**
1. Início	Precisa começar a estudar, mas não sabe por onde.	Ansiedade / Tédio	Sentimento de sobrecarga e falta de direção.	**Interface amigável:** Convida o usuário a começar uma seção de estudos.
2. Foco	Inicia a primeira sessão de estudos.	Expectativa	Tendência a se distrair após 10 minutos.	**Cronômetro Pomodoro:** Inicia ciclo de 25/5.
3. Busca por Motivação	Completa a primeira rodada de estudos.	Satisfação inicial	Necessidade de motivação para continuar o ciclo.	**Gamificação / Seção de Progresso:** Exibe barra de progresso.
4. Manutenção do Ritmo	Vê o progresso diário se acumulando.	Engajamento	Risco de esquecer a revisão dos conteúdos estudados.	***Flashcards* com Repetição Espaçada:** O menu mostra as revisões daquele dia automaticamente.
5. Recaída na Procrastinação	Perde um dia de estudo e volta a se sentir desmotivado.	Culpa / Vontade de desistir	Efeito "que se dane": encarar a falha pontual como o fim do esforço (*what the hell effect*).	**Congelamento de *Streak* / Lote de Revisões:** Perdoa a sequência quebrada e evita mostrar todas as revisões acumuladas de uma vez, reduzindo a sobrecarga do retorno.

Fonte: autoria própria.

Tabela 3 – Jornada de Hazel

Etapa da Jornada	**Ação da Persona (Hazel)**	**Sentimento**	**Ponto de Dor / Desafio**	**Ação do Aplicativo (Funcionalidade)**
1. Planejamento	Tem apenas 2 horas disponíveis para estudar à noite.	Pressa / *Stress*	Perda de tempo tentando decidir o que revisar para maximizar o retorno.	**Integração SRS/Pomodoro:** O aplicativo sugere automaticamente o bloco de conteúdo mais urgente para revisão.
2. Execução	Inicia a revisão do tópico sugerido.	Concentração	Dificuldade em manter a disciplina de revisão devido ao cansaço.	**Cronômetro Pomodoro:** Executa um ciclo de foco seguido de um descanso rápido.
3. Consolidação	Completa as sessões e encerra o dia.	Alívio	Preocupação se o esforço de hoje será retido daqui uma semana.	**Algoritmo SRS:** Atualiza o intervalo de repetição do conteúdo revisado para um período mais longo.
4. Execução após uma semana	Vê no menu a revisão programada.	Confiança	Preocupação se as informações foram esquecidas.	**Algoritmo SRS:** Atualiza o novo intervalo de tempo para a próxima revisão baseado na nota de dificuldade.

Fonte: autoria própria.

Tabela 4 – Jornada de Arabella

Etapa da Jornada	**Ação da Persona (Arabella)**	**Sentimento**	**Ponto de Dor / Desafio**	**Ação do Aplicativo (Funcionalidade)**
1. Avaliação do Humor	Prepara-se para uma longa sessão de revisão.	Fadiga / Ansiedade	Não sabe se está em condições emocionais de absorver conteúdo complexo.	***Check-in* de Saúde:** Solicita ao usuário que avalie seu humor e nível de estresse (escala de 1 a 5).
2. Estudo Adaptado	Começa seus estudos com a sabedoria do seu estado mental.	Aceitação	Iniciar um tópico muito difícil pode causar mais estresse.	**Cronômetro Pomodoro Configurável:** O usuário configura o cronômetro para ciclos mais curtos com pausas mais longas.
3. Revisão	Executa a revisão via *flashcards*.	Foco	Perder tempo procurando o próximo conteúdo a revisar.	**Interface Intuitiva / SRS:** Apresenta o *flashcard* mais crítico na tela de forma clara, com botões visuais para a autoavaliação (fácil/difícil).
4. Reflexão Pós-Sessão	Termina o bloco de estudo.	Alívio	Necessidade de ver o impacto das emoções no desempenho.	**Relatório de *Check-in* e Progresso:** Exibe vários gráficos na tela de progresso a fim do usuário obter uma análise de como sua saúde mental impacta no seu rendimento.

Fonte: autoria própria.

***Brainstorming* de Funcionalidades**

A partir das personas, objetivos e necessidades pode-se começar a etapa de *brainstorming* de funcionalidades. O objetivo desta atividade é gerar um grande número de ideias em um tempo determinado incentivando a criatividade. O processo começa com a releitura das Jornadas de Usuário, e para cada passo da jornada, os participantes sugerem funcionalidades que ajudariam a corrigir os problemas. A seguir está representado o quadro com todas as ideias iniciais.

Figura 12 – Quadro do *Brainstorming* de Funcionalidades



Fonte: autoria própria.

Revisão Técnica, de Negócio e de Experiência do Usuário

A revisão de negócio atua como o primeiro filtro, questionando o valor de cada funcionalidade sob a visão dos objetivos de negócio definidos anteriormente. A análise é feita inserindo de um a três símbolos de dinheiro em cada ideia, conforme seu nível técnico.

Em seguida, a revisão técnica e a revisão de experiência do usuário (UX) agem como filtros de viabilidade e desejabilidade, respectivamente. A revisão técnica avalia a complexidade e o esforço de implementação de cada funcionalidade, considerando os recursos disponíveis. Para medir, deve-se inserir de um a três símbolos de esforço (letra E) conforme o nível do esforço em cada ideia do brainstormming de funcionalidades. Já a revisão de experiência do usuário (UX) avalia se a funcionalidade é simples de usar e se ela resolve o problema da Persona de forma eficaz. Da mesma forma, a medição é feita inserindo de um a três símbolos de coração em cada ideia, conforme seus níveis.

Após essa avaliação, deve-se agrupar essas ideias com suas avaliações em áreas específicas conforme sua agregação ao escopo do projeto, no chamado Gráfico do Semáforo, segundo Caroli (2017). Onde, as ideias agrupadas na zona crítica, três quadrantes do canto inferior esquerdo, serão excluídas do escopo do MVP enquanto as demais permanecem. A seguir está representado o Gráfico do Semáforo deste trabalho.

Figura 13 – Gráfico do Semáforo



Fonte: autoria própria.

Sequenciador

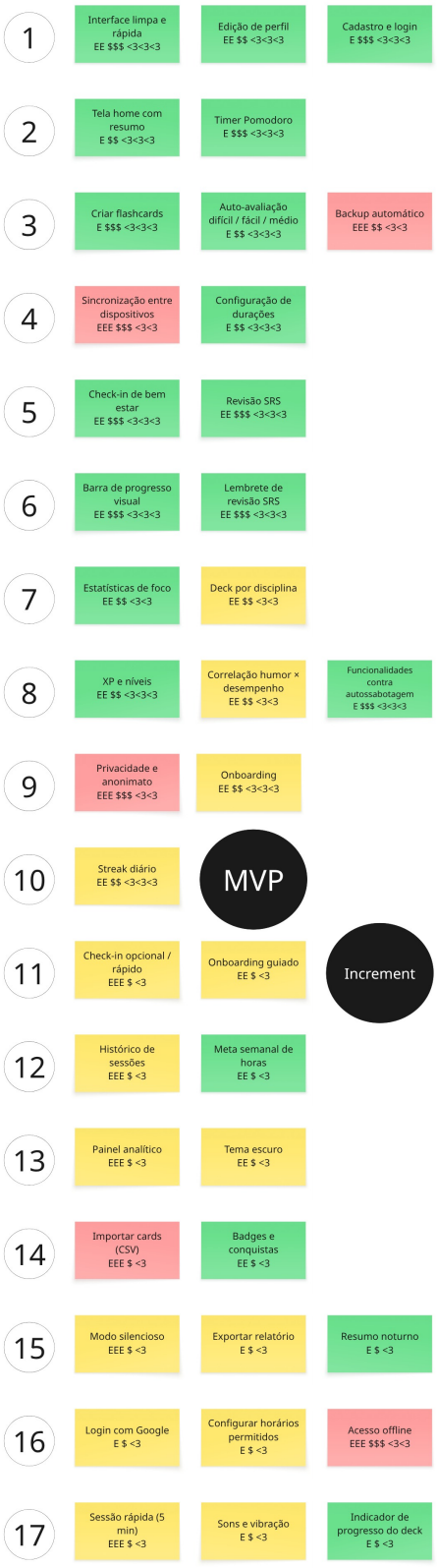
O sequenciador é a ferramenta do *Lean Inception* responsável por dizer a ordem de implementação das funcionalidades, alocando-as em uma linha do tempo composta por ondas. O propósito dessa organização é garantir que o trabalho seja distribuído de maneira incremental e equilibrada, prevenindo a sobrecarga do desenvolvedor e assegurando que o produto evolua de forma sustentável. Conforme Caroli (2017), o sequenciamento não é aleatório, ele deve respeitar regras lógicas que levam em conta o valor entregue ao usuário e sua complexidade técnica para cada item.

Para assegurar o equilíbrio, cada onda de desenvolvimento deve aderir a alguns critérios de formação. Primeiramente, deve respeitar o limite de três funcionalidades por onda. Também não se pode ter uma onda composta exclusivamente por itens de alto esforço e somente pode haver um cartão vermelho por ciclo, este representa funcionalidades

que sua implementação é desconhecida. Além disso, a soma total do esforço técnico por onda deve ser limitada a 5 pontos, garantindo que o volume de trabalho seja adequado.

Cada onda deve atingir uma soma mínima de 4 pontos tanto para o valor de negócio quanto para a experiência do usuário (UX), buscando garantir que todos os ciclos tragam utilidade para o cliente final. Finalmente, a metodologia exige atenção às funcionalidades que possuem pré requisitos de outras funcionalidades, ou seja uma funcionalidade que depende de outra deve ser implementada em uma onda posterior.

Figura 14 – Sequenciador



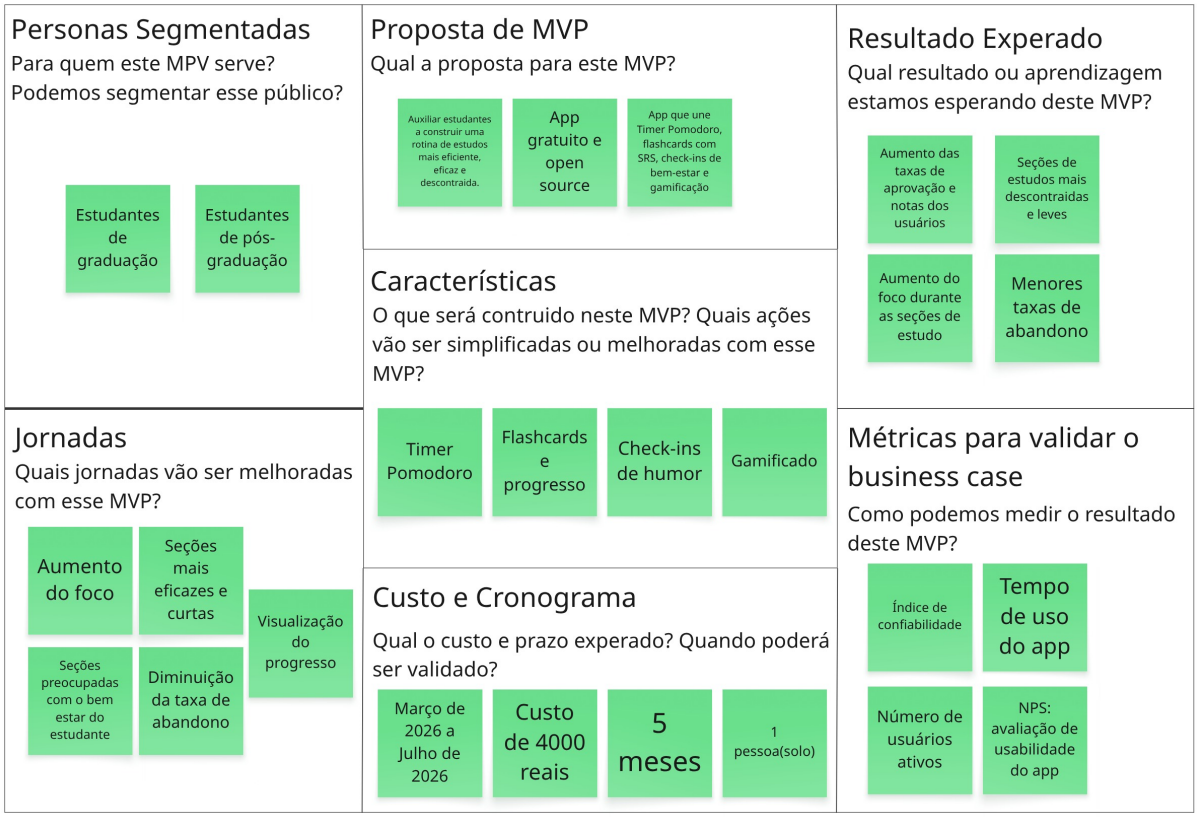
Fonte: autoria própria.

***Canvas* MVP**

Finalmente, a última etapa do *Lean Inception* é a elaboração do *Canvas* MVP,

que é um quadro visual que resume as decisões das etapas anteriores para garantir o alinhamento sobre o que será construído e como prosseguir. De acordo com Caroli (2017), este artefato resume o valor do MVP em sete componentes fundamentais: a definição da proposta do MVP, a identificação das personas atendidas e suas respectivas jornadas, a listagem das funcionalidades prioritárias, os resultados esperados e as métricas para a validação das hipóteses de negócio, finalizando com a estimativa de custos e o cronograma previsto. A seguir está representado o canvas MVP deste trabalho.

Figura 15 – *Canvas* MVP



Fonte: autoria própria.

3.3 Requisitos

Os requisitos são especificações do sistema que devem suprir as necessidades dos usuários, sendo a sua elicitação uma etapa fundamental para o desenvolvimento do *software*. De acordo com Khalid et al. (2025), requisitos mal gerenciados são uma das principais causas de falha em projetos de *software* e aumento de custos, o que torna a engenharia de requisitos uma fase crítica para garantir a qualidade e a satisfação do usuário final.

Sendo assim, para este projeto, foram utilizados os artefatos do *Lean Inception* para elicitar os requisitos. Além disso, utilizou-se o método MoSCoW para a priorização das

funcionalidades levantadas, proporcionando uma visão do que compõe o MVP e apoiando o foco do desenvolvimento nas funcionalidades de maior prioridade.

3.3.1 Requisitos Funcionais

Os requisitos funcionais representam as funcionalidades que o sistema deve oferecer para atender às necessidades dos usuários. Eles descrevem comportamentos, serviços e regras de negócio que o sistema deverá executar.

Histórias de Usuário

As histórias de usuário têm como seu principal objetivo descrever as funcionalidades necessárias do ponto de vista do usuário. Para este projeto, elas foram divididas em épicos, grandes conjuntos de funcionalidades, e *features*, funcionalidades específicas derivadas desses épicos.

A fim de proporcionar maior controle sobre o escopo do MVP, utilizou-se o método de priorização MoSCoW, que classifica os requisitos nas seguintes categorias:

- ***Must* (Deve ter):** requisitos obrigatórios para o funcionamento do MVP;
- ***Should* (Deveria ter):** requisitos importantes, mas que podem ser postergados;
- ***Could* (Poderia ter):** requisitos desejáveis, mas não essenciais;
- ***Won't* (Não terá):** requisitos que não serão implementados nesta versão.

A seguir, apresentam-se as funcionalidades desenvolvidas usando histórias de usuário (US) e priorizadas com MoSCoW.

Épico 1 — Gerenciamento de Usuário e Perfil

- ***Feature* F01: Registro e Acesso.**
  - *US01:* Eu, como usuário, desejo me cadastrar informando nome, e-mail e senha, e realizar *login* no aplicativo, para que meu progresso de estudo seja salvo e acessível de qualquer dispositivo.
  - *Prioridade:* deve ter.
- ***Feature* F02: Edição de Perfil.**
  - *US02:* Eu, como usuário, desejo editar meu nome no aplicativo, para que minhas informações pessoais estejam sempre atualizadas.
  - *Prioridade:* deveria ter.

Épico 2 — Gestão do Ciclo de Estudo

- ***Feature* F03: *Timer* Pomodoro.**
  - *US03:* Eu, como usuário, desejo iniciar um cronômetro de foco e descanso configurável (duração e número de ciclos) para gerenciar meu tempo de estudo com a Técnica Pomodoro.
  - *Prioridade:* deve ter.
- ***Feature* F04: Algoritmo de Repetição Espaçada (SRS).**
  - *US04:* Eu, como usuário, desejo avaliar a dificuldade de um *flashcard* revisado (Fácil, Difícil, Errei, etc.) para que o sistema agende automaticamente a próxima revisão com base no meu desempenho.
  - *Prioridade:* deve ter.
- ***Feature* F05: Listagem de Revisões Pendentes.**
  - *US05:* Eu, como usuário, desejo visualizar na tela inicial (*Dashboard*) quais *flashcards* precisam ser revisados hoje conforme o cronograma SRS, para que eu organize minha sessão de estudos.
  - *Prioridade:* deve ter.

Épico 3 — Gerenciamento de *Flashcards*

- ***Feature* F06: Criação, Edição e Exclusão de *Flashcards*.**
  - *US06:* Eu, como usuário, desejo criar *flashcards* com frente (pergunta) e verso (resposta), bem como editar e excluir *cards* existentes, para que eu possa montar meu próprio banco de conteúdo de estudo.
  - *Prioridade:* deve ter.
- ***Feature* F07: Sessão de Revisão de *Flashcards*.**
  - *US07:* Eu, como usuário, desejo iniciar uma sessão de revisão onde os *flashcards* são exibidos um a um (frente → virar → verso → avaliar dificuldade).
  - *Prioridade:* deve ter.
- ***Feature* F15: Organização de *Flashcards* por Disciplina.**
  - *US15:* Eu, como usuário, desejo agrupar meus *flashcards* em disciplinas (*decks*), para organizar meu conteúdo de estudo por área ou matéria e revisar de forma mais direcionada.

  - *Prioridade:* deveria ter.

Épico 4 — Monitoramento de Bem-Estar e Saúde

- ***Feature* F08: *Check-in* de Saúde.**
  - *US08:* Eu, como usuário, desejo registrar diariamente meu nível de humor e estresse por meio de um *check-in* rápido no *Dashboard*, para monitorar meu bem-estar emocional ao longo dos estudos.
  - *Prioridade:* deve ter.
- ***Feature* F09: Relatórios de Bem-Estar.**
  - *US09:* Eu, como usuário, desejo visualizar na tela de Progresso gráficos que mostrem a evolução do meu humor e seu relacionamento com meu desempenho nos *flashcards* ao longo do tempo.
  - *Prioridade:* poderia ter.

Épico 5 — Gamificação e Engajamento

- ***Feature* F10: Sistema de XP, Níveis e *Streaks*.**
  - *US10:* Eu, como usuário, desejo ganhar pontos de experiência (XP) ao completar ciclos Pomodoro e sessões de revisão de *flashcards*, e visualizar meu nível atual e sequência de dias (*streak*) na tela de Perfil.
  - *Prioridade:* poderia ter.
- ***Feature* F11: Visualização de Progresso.**
  - *US11:* Eu, como usuário, desejo visualizar na tela de Progresso gráficos e indicadores sobre meu desempenho.
  - *Prioridade:* deveria ter.
- ***Feature* F12: Congelamento de *Streak*.**
  - *US12:* Eu, como usuário, desejo que minha sequência de dias (*streak*) seja perdoada uma vez por semana quando eu perder um dia de estudo, para que uma falha pontual não me faça desistir do meu progresso acumulado.
  - *Prioridade:* poderia ter.
- ***Feature* F13: Limitação do Lote de Revisões Pendentes.**

  - *US13:* Eu, como usuário, desejo que a lista de *flashcards* pendentes de revisão seja exibida em lotes limitados por sessão, em vez de todas de uma vez, para não me sentir sobrecarregado ao retomar os estudos após um período de ausência.
  - *Prioridade:* deveria ter.

- ***Feature* F14: Alertas de Intervenção de Bem-Estar.**
  - *US14:* Eu, como usuário, desejo receber um alerta com uma sugestão de ação quando meus *check-ins* de humor ou estresse indicarem um padrão negativo recorrente, para que eu possa ajustar minha rotina antes de me sentir sobrecarregado.
  - *Prioridade:* poderia ter.

Além dos épicos elicitados durante o *Lean Inception*, dois incrementos foram adicionados ao MVP após a sua entrega inicial, a partir de testes práticos e do uso real do aplicativo pelo autor. Diferentemente dos demais requisitos deste trabalho, esses dois não derivam diretamente do *Canvas* MVP original, mas foram priorizados e implementados posteriormente por apresentarem alto valor percebido em relação ao esforço de implementação necessário.

Épico 6 — Incrementos Pós-MVP

- ***Feature* F16: Importação de *Flashcards* via JSON.**
  - *US16:* Eu, como usuário, desejo colar um texto no formato JSON contendo vários *flashcards* já formatados para que sejam criados de uma só vez, em vez de cadastrar cada um manualmente, para que eu possa popular uma disciplina rapidamente com o auxílio de uma IA generativa externa.
  - *Prioridade:* poderia ter.

- ***Feature* F17: Treino Livre.**
  - *US17:* Eu, como usuário, desejo poder revisar *flashcards* adicionais além do lote de revisões pendentes do dia, quando eu já tiver cumprido minha meta diária ou não houver nenhuma revisão vencida, para que eu possa continuar estudando caso tenha tempo e disposição extras.
  - *Prioridade:* poderia ter.

A importação de *flashcards* via JSON (F16) resolve um gargalo observado durante os testes do aplicativo: a criação manual de um volume significativo de *flashcards*, campo a campo, é uma tarefa repetitiva e demorada, sobretudo ao povoar uma disciplina nova

do zero. Para contornar essa barreira sem exigir nenhuma integração direta com serviços de Inteligência Artificial – o que adicionaria custos e complexidade de infraestrutura ao *backend* –, o aplicativo oferece um *prompt* pronto que o usuário pode copiar e enviar a uma IA generativa externa de sua preferência (como o Gemini ou o ChatGPT), bastando especificar o tema, a quantidade e a dificuldade desejados. A resposta da IA, já formatada como uma lista JSON de perguntas e respostas, é colada de volta no aplicativo, e todos os *flashcards* são criados em lote numa única operação transacional no *backend*. Essa abordagem aproveita a disseminação do acesso a ferramentas de IA generativa entre o público universitário sem comprometer o escopo gratuito e de código aberto do projeto, justificando sua priorização mesmo sem ter feito parte do escopo original do MVP.

Já o treino livre (F17) complementa o mecanismo de limitação do lote de revisões pendentes (F13, Seção 2.11) sem reabrir o risco de sobrecarga que esse mecanismo foi projetado para evitar. Enquanto o lote diário é limitado para proteger o usuário do efeito desencorajador de acumular dezenas de revisões vencidas, essa mesma limitação pode frustrar usuários motivados que já cumpriram sua meta do dia e desejam continuar estudando. O treino livre oferece, nesses casos, uma via opcional de continuidade: o usuário pode revisar *flashcards* adicionais – priorizados pelos que estão mais próximos de vencer –, sempre recalculando o algoritmo de repetição espaçada normalmente a cada resposta, exatamente como ocorre no fluxo principal de revisão. Dessa forma, a funcionalidade preserva o propósito protetivo do lote limitado ao mesmo tempo em que concede agência ao usuário que deseja ir além dele.

Portanto, a seguir estão representados todos os requisitos funcionais deste trabalho.

Tabela 5 – Requisitos Funcionais

ID	**Descrição**
FUNC01	Cadastro e autenticação de usuário.
FUNC02	Edição de dados cadastrais do perfil.
FUNC03	Gerenciamento de ciclos de foco e descanso configuráveis.
FUNC04	Criação, edição e exclusão de *flashcards* com frente e verso.
FUNC05	Sessão de revisão de *flashcards* com *flip* de carta e avaliação de dificuldade.
FUNC06	Agendamento automático de revisões baseado no algoritmo SRS e no *feedback* de dificuldade.
FUNC07	Listagem de *flashcards* pendentes de revisão.
FUNC08	Registro de *check-in* de humor e nível de estresse.
FUNC09	Visualização de estatísticas de desempenho.
FUNC10	Visualização de dados de bem-estar.
FUNC11	Sistema de gamificação com pontuação por XP, níveis e *streak* de dias consecutivos.
FUNC12	Congelamento semanal de *streak* em caso de dia de estudo perdido.
FUNC13	Limitação do lote de *flashcards* pendentes exibidos por sessão de revisão.
FUNC14	Emissão de alertas de intervenção a partir de padrões negativos recorrentes nos *check-ins* de bem-estar.
FUNC15	Agrupamento de *flashcards* em disciplinas (*decks*).
FUNC16	Importação em lote de *flashcards* a partir de um texto colado em formato JSON.
FUNC17	Treino livre de *flashcards* adicionais após o cumprimento da meta diária de revisões.

Fonte: autoria própria.

Para tornar explícita a relação entre as necessidades levantadas junto às proto-personas e os artefatos de requisitos, a Tabela 6 apresenta a matriz de rastreabilidade entre as histórias de usuário, as *features* correspondentes, os requisitos funcionais derivados e a prioridade MoSCoW final de cada um.

Tabela 6 – Matriz de Rastreabilidade entre Histórias de Usuário e Requisitos Funcionais

Épico	**US**	***Feature***	**Requisito**	**MoSCoW**
1	US01	F01 – Registro e Acesso	FUNC01	Deve ter
1	US02	F02 – Edição de Perfil	FUNC02	Deveria ter
2	US03	F03 – *Timer* Pomodoro	FUNC03	Deve ter
2	US04	F04 – Algoritmo de Repetição Espaçada (SRS)	FUNC06	Deve ter
2	US05	F05 – Listagem de Revisões Pendentes	FUNC07	Deve ter
3	US06	F06 – Criação, Edição e Exclusão de *Flashcards*	FUNC04	Deve ter
3	US07	F07 – Sessão de Revisão de *Flashcards*	FUNC05	Deve ter
3	US15	F15 – Organização de *Flashcards* por Disciplina	FUNC15	Deveria ter
4	US08	F08 – *Check-in* de Saúde	FUNC08	Deve ter
4	US09	F09 – Relatórios de Bem-Estar	FUNC10	Poderia ter
5	US10	F10 – Sistema de XP, Níveis e *Streaks*	FUNC11	Poderia ter
5	US11	F11 – Visualização de Progresso	FUNC09	Deveria ter
5	US12	F12 – Congelamento de *Streak*	FUNC12	Poderia ter
5	US13	F13 – Limitação do Lote de Revisões Pendentes	FUNC13	Deveria ter
5	US14	F14 – Alertas de Intervenção de Bem-Estar	FUNC14	Poderia ter
6	US16	F16 – Importação de *Flashcards* via JSON	FUNC16	Poderia ter
6	US17	F17 – Treino Livre	FUNC17	Poderia ter

Fonte: autoria própria.

3.3.2 Requisitos Não Funcionais

Os requisitos não funcionais são especificações que não se referem às funcionalidades específicas do sistema, mas sim a como o *software* deve funcionar e seu desempenho. Segundo Sommerville (2011), esses requisitos descrevem restrições e propriedades do sistema, como desempenho, segurança e usabilidade, sendo essenciais para a aceitação do produto. Dessa maneira, Gobov e Zuieva (2025) destacam que os atributos de qualidade de *software* são determinantes para o valor do produto final, e a falta desses requisitos durante a engenharia de *software* é uma das causas primárias de insatisfação do usuário e de falhas.

FURPS+

Para organizar os requisitos não funcionais do projeto, utilizou-se o modelo FURPS+. Esse modelo, adotado no Rational Unified Process, categoriza os atributos de qualidade em seis tópicos:

- ***Functionality*:** Trata dos requisitos funcionais, já abordados.

- ***Usability*:** Refere-se à facilidade de uso, *design* da interface e experiência do usuário (UX). Segundo Gobov e Zuieva (2025), a usabilidade é um atributo externo crítico que impacta diretamente a produtividade do usuário.

- ***Reliability*:** Trata da confiabilidade, incluindo frequência de falhas, capacidade de recuperação e integridade dos dados.

- ***Performance*:** Refere-se a eficiência do sistema, como tempo de resposta e consumo de recursos.

- ***Supportability*:** Refere-se a facilidade de manutenção, testabilidade e portabilidade do *software*.

- ***Plus*:** Inclui restrições de implementação, interface, requisitos físicos e observações legais.

A partir das métricas do modelo, os requisitos não funcionais do aplicativo foram identificados e classificados na Tabela 7.

Tabela 7 – Requisitos Não Funcionais

ID	**Descrição**
USAB01	A interface deve seguir uma interface minimalista que garante consistência visual e redução da carga cognitiva.
USAB02	O aplicativo deve fornecer *feedback* visual claro ao iniciar, pausar e finalizar ciclos Pomodoro.
USAB03	O sistema deve seguir padrões de acessibilidade para garantir o uso por diferentes perfis de usuários.
USAB04	O fluxo completo de resposta a um *flashcard* deve ser executável com no máximo dois toques na tela.
USAB05	O *check-in* de humor deve ser concluído em no máximo três interações a partir do *Dashboard*.
CONF01	O sistema deve garantir a persistência dos dados do usuário mesmo após reinicialização do dispositivo.
CONF02	O aplicativo deve estar em conformidade com a Lei Geral de Proteção de Dados (LGPD).
CONF03	O sistema deve garantir disponibilidade mínima de 99% do tempo.
PERF01	O acionamento e a pausa do cronômetro Pomodoro devem ser instantâneos (tempo de resposta < 500ms).
PERF02	O cálculo do próximo agendamento pelo algoritmo SRS não deve exceder 1 segundo por *flashcard* avaliado.
PERF03	O aplicativo deve sincronizar dados com o servidor em menos de 5 segundos sob conexão estável.
SUPO01	O aplicativo deve operar no sistema operacional Android.
SUPO02	O código fonte deve ser documentado, modular e organizado.
SUPO03	O sistema deve suportar atualizações do banco de dados sem necessidade de reinstalação do aplicativo.
PLUS01	O aplicativo deve ocupar menos de 100 MB de armazenamento no dispositivo.

Fonte: autoria própria.

4 Desenvolvimento

Este capítulo mostra as decisões e os artefatos produzidos para o desenvolvimento do projeto. É discutido o suporte tecnológico, a arquitetura do sistema, a representação visual, diagramas, o projeto de dados, a estratégia de testes de *software*, a prototipação de alta fidelidade e a implantação do aplicativo em produção.

4.1 Suporte Tecnológico

Esta seção apresenta as ferramentas utilizadas ao longo de todo o desenvolvimento do *software*, desde o planejamento até a implementação do MVP. As ferramentas foram escolhidas visando a metodologia ágil do desenvolvimento do MVP e a experiência prévia com as mesmas.

4.1.1 Miro

A ferramenta Miro permite criar diagramas ilustrativos e exportá-los em imagens. Além disso, a aplicação da metodologia *Lean Inception* exige um ambiente altamente colaborativo para a construção de artefatos como o *Canvas* MVP. O Miro foi utilizado para ilustrar os processos do *Lean Inception*, facilitando a visualização do produto e o alinhamento das expectativas.

4.1.2 Figma

O Figma é a ferramenta escolhida para a elaboração dos protótipos de alta fidelidade. Sua relevância está na capacidade de criar componentes reutilizáveis e fluxos de interação que simulam a experiência real do usuário. Portanto, o Figma é útil para validar a interface e testar a usabilidade das funcionalidades antes da codificação (FIGMA, 2025).

4.1.3 Draw.io

O Draw.io foi utilizado para a modelagem dos diagramas de classe e diagramas de pacotes. A ferramenta permite documentar a arquitetura lógica do aplicativo, essencial para garantir que a integração entre os componentes e o banco de dados seja implementada de forma correta (DRAW.IO, 2025).

4.1.4 React Native e Expo

Para o desenvolvimento, foi utilizado o React Native junto com o *framework* Expo. Essa escolha justifica-se pela necessidade de uma aplicação *mobile* (Android) que utilize uma base de código única. O Expo acelera o desenvolvimento ao fornecer APIs prontas para notificações e acesso ao *hardware*, além de permitir testes em tempo real em dispositivos físicos através do Expo Go (EXPO, 2025).

4.1.5 Node.js e Express

Como tecnologia de *backend*, foi utilizado o Node.js com o *framework* Express. Este conjunto tecnológico é reconhecido pela sua escalabilidade e alta performance em aplicações com bancos de dados, sendo ideal para gerenciar as requisições de *check-ins* de saúde e o processamento do algoritmo de repetição espaçada em tempo real.

4.1.6 PostgreSQL

Para a persistência dos dados, foi utilizado o PostgreSQL como o Sistema Gerenciador de Banco de Dados Relacional do aplicativo. Suportando consultas complexas e garantindo a integridade conforme as normas da LGPD.

4.1.7 Git, GitHub e Licenciamento

O controle de versionamento do código foi realizado utilizando Git, com o armazenamento dos repositórios no GitHub. Essa prática apoia a manutenção do histórico do código, permitindo o gerenciamento de ramos para novas funcionalidades e contribuindo para a rastreabilidade e a integridade do código ao longo do ciclo de desenvolvimento (GITHUB, 2025). A documentação deste trabalho está disponível no repositório público <https://github.com/H0lzz/TCC1.2>, enquanto o código-fonte do aplicativo Focus está organizado em dois repositórios próprios, um para o *backend* (<https://github.com/H0lzz/focus-backend>) e outro para o *frontend* (<https://github.com/H0lzz/focus-mobile>).

Por se tratar de um projeto gratuito e *open source*, conforme definido nos objetivos deste trabalho, o código do aplicativo Focus é distribuído sob a licença permissiva MIT. Essa licença permite que qualquer pessoa utilize, copie, modifique e redistribua o código, inclusive para fins comerciais, desde que mantido o aviso de *copyright* original, favorecendo a democratização do acesso às metodologias de estudo implementadas. Cada repositório contém um arquivo de licença (`LICENSE`) e um `README` com instruções de instalação,

configuração do ambiente de desenvolvimento e diretrizes de contribuição, de modo a documentar o projeto para outros desenvolvedores interessados em utilizá-lo ou estendê-lo.

4.1.8 Trello

Para o gerenciamento ágil do projeto, o Trello foi utilizado como a ferramenta do quadro do Kanban. Ele permitiu o acompanhamento das tarefas elicitadas no Sequenciador do *Lean Inception*, facilitando a organização das *sprints* de desenvolvimento e a visualização do progresso das *features* do MVP em tempo real.

4.1.9 BrModelo

A ferramenta BrModelo foi utilizada para fazer o projeto de dados (diagrama entidade-relacionamento e diagrama lógico de dados). Através dessa ferramenta, realiza-se a modelagem conceitual e lógica do sistema.

4.1.10 Claude (Assistente de IA)

O Claude, assistente de inteligência artificial desenvolvido pela Anthropic, foi utilizado como ferramenta de apoio ao longo de todo o desenvolvimento deste trabalho, por meio da interface de linha de comando Claude Code (ANTHROPIC, 2026).

No desenvolvimento do código-fonte, a ferramenta auxiliou na implementação de funcionalidades do *backend* e do *frontend*, na depuração de problemas (como falhas de configuração do ambiente, migrações do banco de dados e condições de corrida na sincronização de estado da aplicação *mobile*) e na correção de problemas de usabilidade identificados durante os testes manuais do aplicativo. No apoio aos testes, a ferramenta colaborou na escrita dos testes unitários automatizados da camada de serviços do *backend*, discutidos na Seção 4.5. No levantamento do referencial teórico, o Claude auxiliou na leitura dos artigos selecionados na base Scopus e na síntese de suas contribuições, apoiando a identificação de evidências relevantes para fundamentar as funcionalidades implementadas no aplicativo. Por fim, na redação deste documento, a ferramenta colaborou na construção argumentativa do texto, auxiliando na estruturação, revisão e encadeamento das ideias entre as seções. Em todas essas frentes, o uso da ferramenta ocorreu sob supervisão direta do autor, responsável pela validação técnica, pela verificação da fidelidade das afirmações em relação à implementação real do projeto e pela decisão final sobre todo o conteúdo produzido.

4.2 Arquitetura

Segundo Pompeo, Tucci e Hoorn (2025), a arquitetura é uma abstração de alto nível que funciona como um guia para a implementação, sendo essencial para os aspectos de qualidade, como performance, confiabilidade e manutenibilidade. Dessa forma, o modelo adotado busca equilibrar as necessidades de negócio e os objetivos técnicos do sistema. Foi definido que um modelo de arquitetura em camadas, com processamento centralizado em um *backend cliente-servidor*, seria seguido no desenvolvimento, decisão que foi mantida até a implementação final. Essa escolha se justifica pela necessidade de centralizar, em um único local, o cálculo do algoritmo de repetição espaçada e a persistência consistente dos dados de progresso e de *check-in* entre diferentes dispositivos do mesmo usuário, o que seria mais difícil de garantir em uma arquitetura totalmente local ao dispositivo. Essa opção também é favorecida pela delimitação do público-alvo a estudantes universitários, que, ao contrário de um público mais heterogêneo, tendem a ter acesso a conectividade estável durante boa parte da rotina de estudos (redes *wi-fi* do campus, bibliotecas e residências), reduzindo o impacto da ausência de um modo *offline* nesta primeira versão do produto.

4.2.1 Padrão Arquitetural em Camadas

A definição da estrutura arquitetural visa mitigar riscos. Conforme destacado por Khalid et al. (2025), a má gestão de requisitos e de sua tradução técnica é uma causa primária de falhas em projetos. Para evitar esse cenário, adotou-se uma organização em camadas que isola as preocupações de interface, lógica de negócio e persistência de dados.

Esta separação permite que a arquitetura facilite a evolução do código sem comprometer a estabilidade do sistema (POMPEO; TUCCI; HOORN, 2025). As camadas são divididas em:

- **Apresentação (*View*):** Interface móvel que interage com o usuário.
- **Processamento (*Controller*/*Service*):** Onde as funções são validadas e executadas.
- **Acesso a Dados (*Repository*/*Model*):** Camada que abstrai a complexidade do banco de dados relacional.

4.2.2 *Backend*

O *backend*, desenvolvido em Node.js, atua como o núcleo da aplicação. De acordo com Pompeo, Tucci e Hoorn (2025), a especificação cuidadosa do *design* arquitetural

permite fortalecer a confiabilidade do *software*. Dessa forma, a lógica de agendamento de revisões e os cálculos são centralizados no servidor para garantir a integridade dos dados e o cumprimento dos requisitos de qualidade.

A organização interna do *backend* foi estruturada para suportar atributos de qualidade como a manutenibilidade, distribuindo as responsabilidades da seguinte forma:

- ***Controllers*:** Responsáveis por receber e responder às requisições da interface.
- ***Services*:** Camada onde reside a lógica de domínio, isolada de interferências externas.
- ***Repositories*:** Camada dedicada exclusivamente à interação com o SGBDR PostgreSQL.

4.2.3 API RESTful, Segurança e Privacidade

A comunicação entre o cliente e o servidor é estabelecida através de uma API RESTful baseada no protocolo HTTP e no formato JSON. Essa escolha agrega performance conforme dito por Gobov e Zuieva (2025) ao discutirem as características de eficiência de tempo. Para apoiar a confidencialidade e a integridade das informações dos usuários, a arquitetura adota autenticação via *JSON Web Token* (JWT) com expiração configurável, alinhando-se às boas práticas de segurança mencionadas por Pompeo, Tucci e Hoorn (2025) como técnicas para preservação da qualidade arquitetural.

A privacidade dos usuários é um requisito central deste projeto, dada a natureza sensível dos dados de *check-in* de humor e estresse coletados pelo aplicativo. Em conformidade com a LGPD (BRASIL, 2018), foram adotadas as seguintes medidas técnicas: (i) em produção, toda comunicação entre cliente e servidor ocorre via HTTPS/TLS, protegendo os dados em trânsito; (ii) as senhas dos usuários são armazenadas com funções de *hash* criptográfico (nunca em texto plano); (iii) a coleta de dados é limitada ao estritamente necessário para o funcionamento das funcionalidades descritas nos requisitos (princípio da minimização de dados); e (iv) o usuário pode solicitar, a partir da tela de Perfil, a exportação de todos os seus dados pessoais em formato JSON ou a exclusão definitiva da conta, mediante confirmação de senha, atendendo aos direitos previstos pela lei. A exclusão é propagada automaticamente a todos os dados relacionados ao usuário (*flashcards*, disciplinas, revisões, *check-ins* e sessões de foco) por meio de restrições de exclusão em cascata (*cascade delete*) definidas no modelo de dados, dispensando rotinas adicionais de limpeza manual.

4.2.4 Mapeamento Objeto-Relacional (ORM)

Para garantir a qualidade na persistência dos dados, foi utilizado o Prisma como mapeamento objeto-relacional (ORM), fazendo a ligação entre o Node.js e o PostgreSQL. Essa camada de abstração é útil para a manutenção, pois permite que a estrutura do banco de dados evolua com baixo impacto na camada de serviços. Além disso, o uso do ORM facilita a implementação dos requisitos técnicos.

4.2.5 *Frontend*

O *frontend* representa a interface do sistema e satisfaz os atributos de usabilidade e experiência do usuário. Segundo Gobov e Zuieva (2025), a usabilidade é um atributo de qualidade externo que determina quanto o usuário final consegue operar a ferramenta. Portanto, a arquitetura do *frontend* foi desenhada para ser intuitiva, reduzindo a dificuldade do usuário durante o uso.

React Native e Expo

A escolha do React Native como base tecnológica permite a criação de componentes de *software* reutilizáveis, cujas propriedades e relacionamentos definem a dinâmica da interface. A utilização do *framework* Expo complementa essa estrutura ao facilitar o acesso a recursos nativos do *hardware* do dispositivo. Essa abordagem assegura que a qualidade do *software*, conforme definida por Khalid et al. (2025), seja mantida independentemente do sistema Android utilizado pelo usuário, otimizando o processo de entrega de valor.

4.3 Representação Visual do Sistema

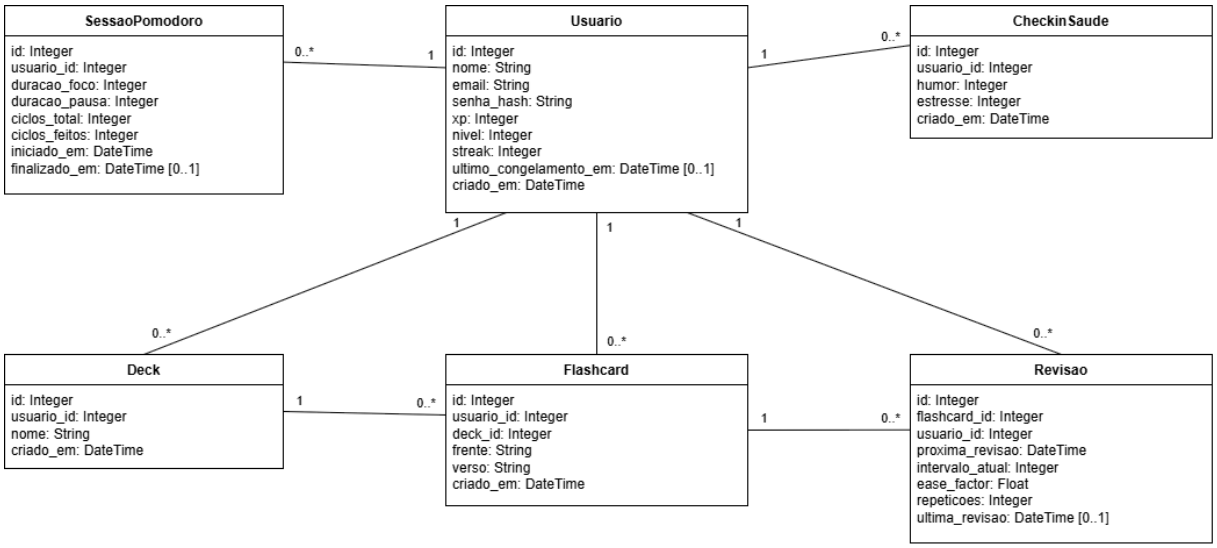
A representação visual do sistema constitui a implementação da arquitetura desenvolvida, traduzindo abstrações conceituais em diagramas técnicos que orientaram o desenvolvimento, a fim de gerenciar a complexidade do *software*. Através de modelos visuais, busca-se estabelecer um modelo que assegura a integridade entre os requisitos de *software* e a estrutura física do código, garantindo que os atributos de qualidade sejam preservados durante todo o ciclo de vida.

4.3.1 Diagrama de Classes

O diagrama de classes do sistema foi estruturado para definir as entidades fundamentais, seus atributos e seus relacionamentos de associação. De acordo com a pesquisa de Kim e Kook (2024), a eficácia de um diagrama de classe está diretamente ligada ao

seu *layout* e controle de complexidade, uma vez que esses fatores impactam significativamente a compreensibilidade do modelo pelo desenvolvedor. Para garantir isso, o diagrama foi organizado seguindo um fluxo lógico de leitura, a seguir está representado o diagrama de classes deste projeto.

Figura 16 – Diagrama de Classes



Fonte: autoria própria.

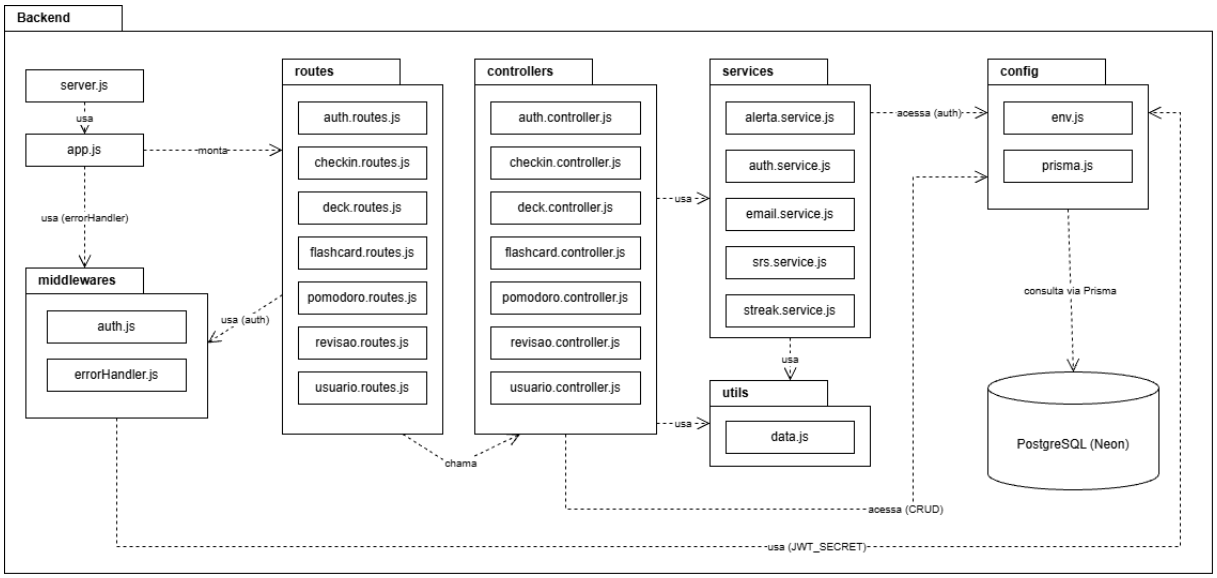
As classes representadas no diagrama não possuem métodos, o que decorre de uma decisão de implementação do *backend*. O código foi escrito em JavaScript seguindo um estilo funcional, organizado em módulos que exportam funções, e não em classes com comportamento encapsulado. Assim, as entidades do diagrama correspondem às estruturas de dados persistidas no PostgreSQL, que apenas armazenam o estado do sistema. O comportamento, por sua vez, está concentrado na camada de serviços, como o cálculo do próximo intervalo de revisão pelo algoritmo de repetição espaçada, a atualização da sequência de dias de estudo e a geração de alertas de bem-estar, e na camada de controladores, responsável por receber as requisições e coordenar essas operações. Por esse motivo, o diagrama de classes foi utilizado como um modelo de domínio, enquanto a organização dos módulos que implementam o comportamento do sistema é apresentada no Diagrama de Pacotes do *Backend* (Figura 17).

4.3.2 Diagrama de Pacotes

Os Diagramas de Pacotes organizam os elementos do sistema em módulos lógicos, isolando as necessidades do *frontend* e do *backend*. No *frontend*, a segmentação em pacotes de interface, lógica e serviços maximiza a manutenibilidade e a usabilidade (GOBOV; ZUIEVA, 2025). No *backend*, a estruturação em camadas de rotas, controladores e serviços reflete o compromisso com o baixo acoplamento e a confiabilidade. Essa or-

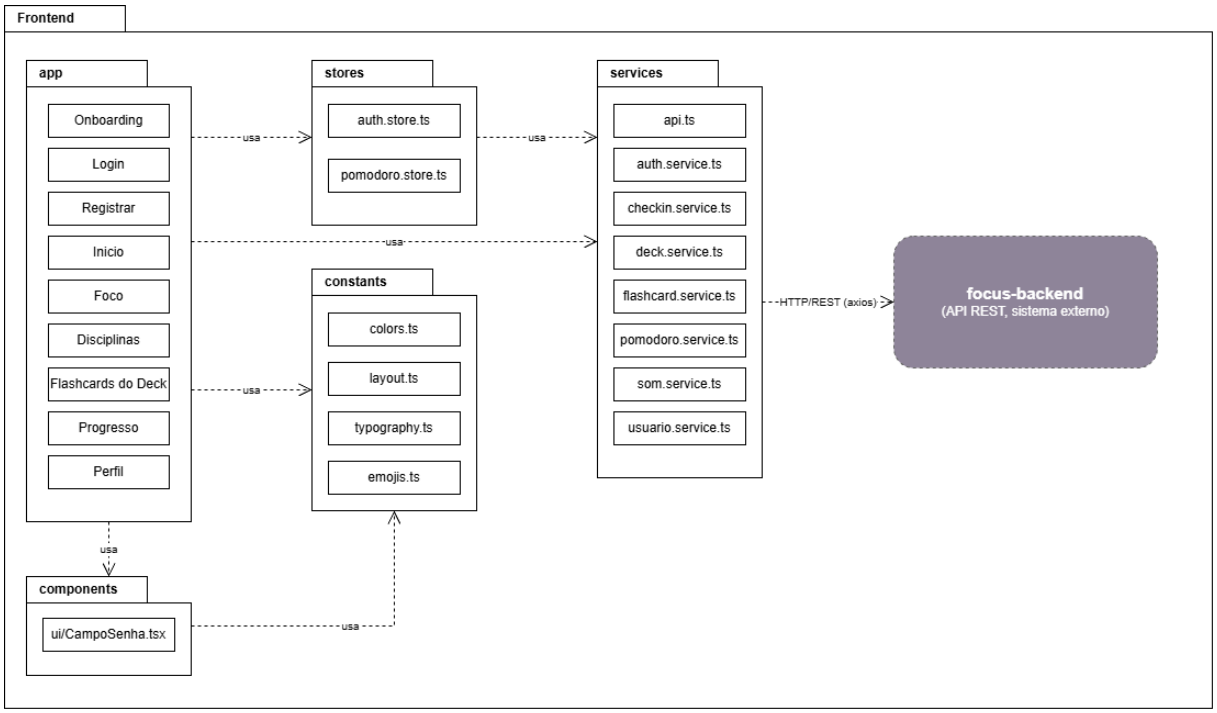
ganização visual é essencial para mitigar falhas na tradução dos requisitos (KHALID et al., 2025), permitindo que cada pacote evolua de forma independente sem comprometer a estabilidade do aplicativo. A seguir estão representados os diagramas de pacotes deste trabalho.

Figura 17 – Diagrama de Pacotes do *Backend*



Fonte: autoria própria.

Figura 18 – Diagrama de Pacotes do *Frontend*



Fonte: autoria própria.

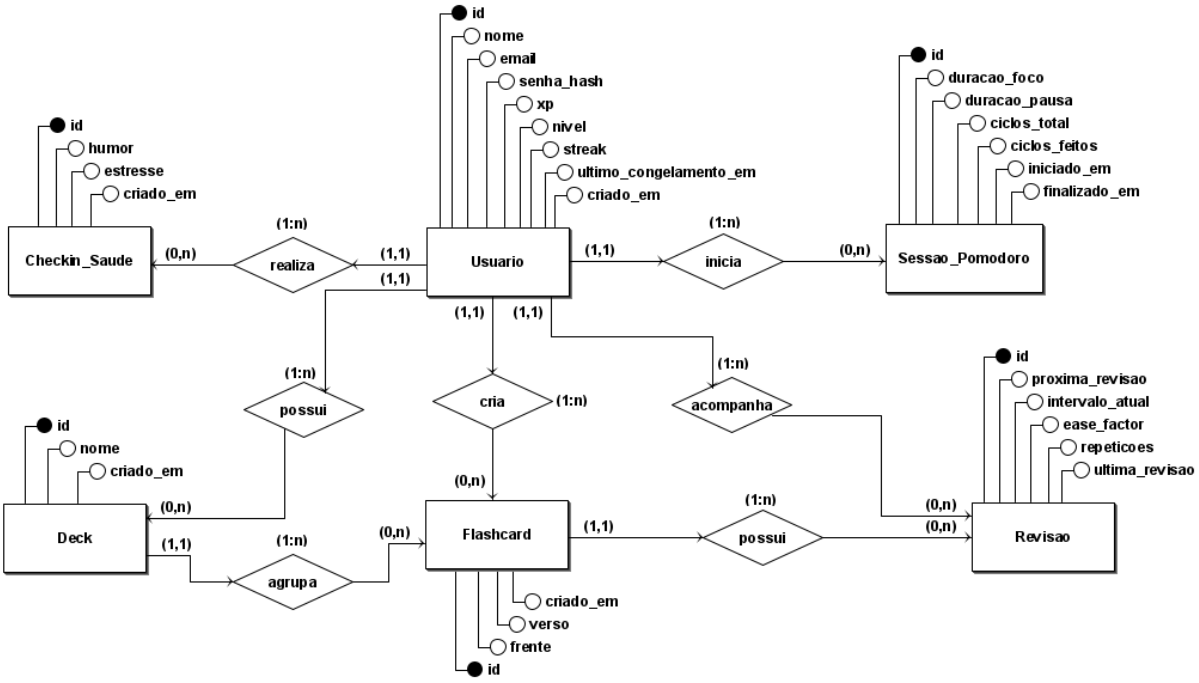
4.4 Projeto de Dados

O Projeto de Dados tem como objetivo modelar a estrutura de persistência dos dados do aplicativo, definindo as entidades, atributos e relacionamentos do mesmo. A modelagem foi realizada em dois níveis: o diagrama entidade-relacionamento, que representa de forma abstrata as entidades e seus relacionamentos, e o diagrama lógico, que traduz essa abstração em uma estrutura compatível com o sistema gerenciador de banco de dados relacional. Todas decisões de modelagem foram guiadas pelos requisitos funcionais e não funcionais levantados anteriormente.

4.4.1 Diagrama Conceitual

A etapa conceitual da modelagem de dados é responsável por abstrair as entidades do sistema, seus atributos e os relacionamentos existentes entre elas, de forma universal e compatível com qualquer tecnologia de banco de dados através do diagrama entidade-relacionamento (DE-R). Essa etapa exige a transformação dos requisitos em um modelo com regras, envolvendo uma abstração maior do que nas próximas etapas, sendo apontada como a principal fonte de dificuldades cognitivas no processo de modelagem (SHIN et al., 2025). Para este trabalho, o DE-R foi construído a partir das entidades identificadas durante o *Lean Inception* e o levantamento de requisitos. A seguir está representado o DE-R deste projeto.

Figura 19 – Diagrama Entidade-Relacionamento (DE-R) do Projeto

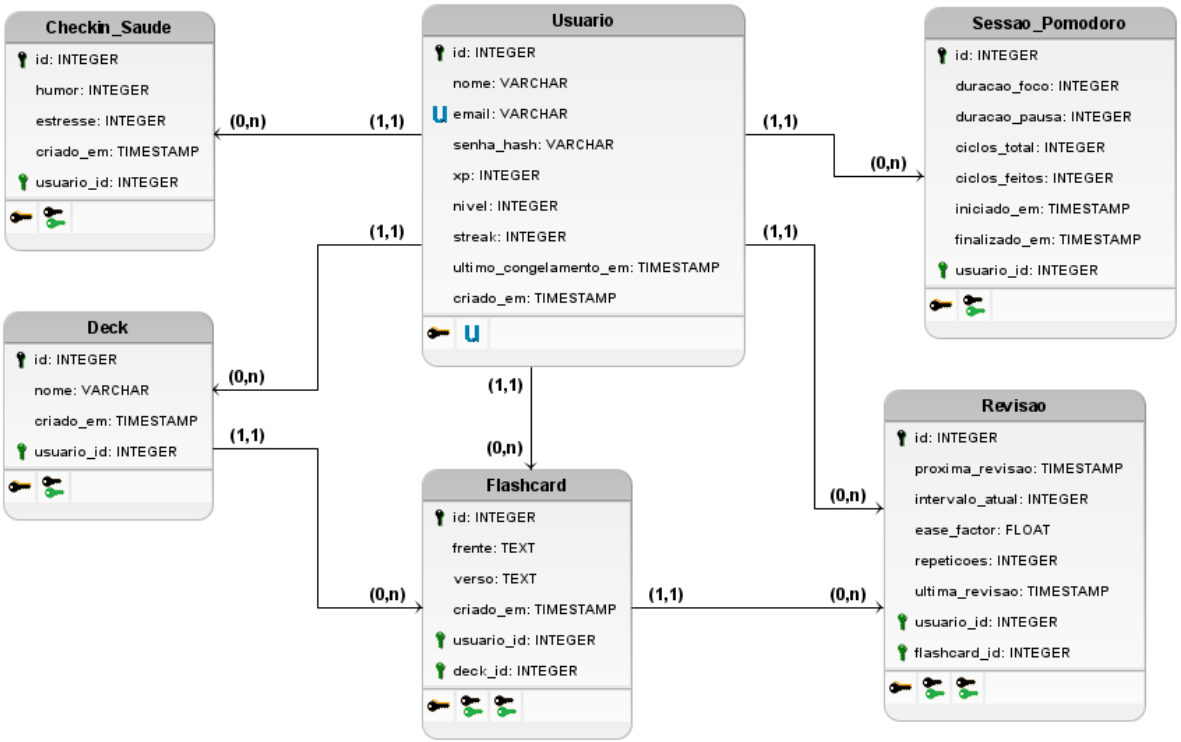


Fonte: autoria própria.

4.4.2 Diagrama Lógico

Já o diagrama lógico representa a segunda etapa da modelagem de dados, sendo responsável por transformar a abstração do DE-R em uma estrutura lógica compatível com o modelo relacional, definindo tabelas, colunas, chaves primárias, chaves estrangeiras e as restrições de integridade referencial entre as entidades. Essa transição do modelo conceitual para o modelo lógico segue um conjunto de regras de mapeamento bem definidas, onde cada entidade do diagrama conceitual é transformada em uma tabela, cada atributo em uma coluna e cada relacionamento em uma chave estrangeira alocada no lado correto da cardinalidade ou em uma nova relação. Esse processo de mudança do modelo entre níveis de abstração é um dos pilares do desenvolvimento orientado a modelos, no qual modelos universais são aperfeiçoados em modelos mais específicos por meio de regras de mapeamento (LO; HUANG, 2012). Para este trabalho, o diagrama lógico foi derivado diretamente do DE-R, respeitando as cardinalidades levantadas e as decisões de modelagem tomadas durante o *Lean Inception*, resultando em um esquema relacional normalizado, pronto para ser implementado no SGBD. A seguir está representado o diagrama lógico deste projeto.

Figura 20 – Diagrama Lógico do Projeto



Fonte: autoria própria.

4.5 Testes de Software

A etapa de testes faz parte do ciclo de vida de desenvolvimento do *software*, sendo definido como o processo que engloba todas as atividades do ciclo de vida, estáticas e dinâmicas, relacionadas ao planejamento, preparação e avaliação do mesmo, com o objetivo de verificar se satisfaz os requisitos especificados, demonstrar que estão aptos para o uso e detectar defeitos (GOERICKE, 2020). A falta dessa etapa representa um risco ao projeto, pois *bugs* são significativamente mais fáceis e baratos de corrigir em estágios iniciais do desenvolvimento do que em fases avançadas ou após a entrega ao usuário final (GOERICKE, 2020).

O desenvolvimento deste projeto adota a prática de TDD como estratégia de qualidade, que desempenha um papel fundamental na definição da estratégia de testes ao longo de todo o desenvolvimento (GOERICKE, 2020). O TDD garante que cada módulo de regra de negócio do aplicativo seja coberto por testes, assegurando que as regras de negócio estejam sempre verificáveis e coerentes aos requisitos.

A estratégia de testes adotada segue a pirâmide de testes, que divide os testes em três: os testes unitários, que podem ser executados em níveis unitários, os testes de integração que são mais lentos pois integram dois ou mais módulos e os testes de sistema que englobam todo o sistema. Dessa forma, os testes unitários, níveis mais baixos, são os mais adequados para automação (GOERICKE, 2020). Seguindo esse princípio, a base da pirâmide foi composta por testes unitários automatizados com o *framework* Jest, cobrindo a totalidade da camada de serviços do *backend* (100% de *statements*, *branches* e funções): o algoritmo de repetição espaçada (SRS), o cálculo de *streak* com congelamento, a geração de alertas de bem-estar, a validação de e-mail e a autenticação de usuários (cadastro e *login*), validando tanto os cálculos de negócio quanto os cenários de erro (credenciais inválidas, e-mail já cadastrado, formatos e domínios inválidos). A automação de testes de integração e de sistema, que exercitam as rotas HTTP e a camada de controladores do *backend* de ponta a ponta, permanece como um trabalho futuro deste projeto.

Para aplicações móveis, a quantidade de módulos de qualidade a serem considerados representam o principal desafio dos testes, englobando a funcionalidade, a usabilidade, o desempenho, a segurança, a interoperabilidade e a confiabilidade (GOERICKE, 2020). Diante disso, como o Focus foi desenvolvido para o sistema Android, não foi possível cobrir todos os dispositivos e versões existentes. Portanto, os testes de usabilidade foram realizados em um número limitado de aparelhos Android, priorizando os modelos mais utilizados pelo público.

4.6 Prototipação

A prototipação é a etapa do desenvolvimento onde as interfaces de usuário são modeladas através de um processo de criação de representações visuais. Essas representações devem mostrar como a interface deve ser e como os usuários devem interagir com ela, com base nos requisitos do *software* (LETAW, 2024). Essa abordagem oferece uma experimentação da interface e a identificação de problemas recentes. Logo, alterar um protótipo de alta fidelidade é mais fácil e rápido do que alterar sua implementação em código (LETAW, 2024).

Conforme Letaw (2024), existem três níveis de protótipos: o de baixa fidelidade, composto por esboços aproximados que permitem coletar *feedback* sobre funcionalidades de alto nível com grande flexibilidade para mudanças; o de média fidelidade, uma ilustração mais detalhada que permite coletar *feedback* sobre pequenas mudanças em funcionalidades já definidas; e o de alta fidelidade, um modelo digital que se assemelha a uma interface pronta, permitindo coletar *feedback* sobre ajustes específicos.

Para este projeto, foi utilizado o Figma como ferramenta de criação de protótipos de alta fidelidade. Portanto, os requisitos funcionais foram validados, reduzindo o risco de retrabalho durante a fase de desenvolvimento do código. O protótipo desenvolvido foi utilizado como base para a implementação das páginas principais do aplicativo. Como o desenvolvimento do MVP já foi concluído e o aplicativo Focus está disponível para instalação e uso direto, capturas de tela do produto final não são incluídas como apêndice deste trabalho — a aplicação prática das decisões de *design* validadas no protótipo pode ser verificada diretamente na execução do aplicativo.

4.7 Implantação em Produção

Como um dos objetivos deste trabalho é disponibilizar o Focus como um produto gratuito e de código aberto, acessível a estudantes reais e não apenas executável em ambiente local de desenvolvimento, foi definida uma estratégia de implantação em produção para o *backend*, o banco de dados e o aplicativo *mobile*. Todos os serviços utilizados foram operados dentro dos limites de suas camadas gratuitas, mantendo o custo de operação do projeto nulo durante o desenvolvimento e a avaliação deste trabalho.

4.7.1 Banco de Dados em Nuvem

Para a hospedagem do banco de dados em produção, foi utilizado o Neon, um serviço de banco de dados PostgreSQL totalmente gerenciado e sem servidor (*serverless*)

(NEON, 2026). Essa escolha mantém compatibilidade total com o Prisma ORM já utilizado no desenvolvimento, exigindo apenas a configuração da variável de ambiente de conexão (`DATABASE_URL`), sem qualquer alteração no código da aplicação. Por se tratar de uma camada gratuita, o Neon suspende a instância de computação após um período de inatividade, reativando-a automaticamente na requisição seguinte, o que introduz uma latência adicional nesse primeiro acesso.

4.7.2 Hospedagem do *Backend*

O *backend* Node.js foi implantado no Render, uma plataforma de hospedagem em nuvem com suporte nativo a aplicações Node.js e integração contínua a partir do repositório Git (RENDER, 2026). A configuração da implantação foi definida de forma declarativa no arquivo `render.yaml`, versionado junto ao código-fonte, especificando o ambiente de execução, os comandos de *build* e de inicialização, e as variáveis de ambiente necessárias, como a chave secreta de assinatura do JWT e a cadeia de conexão com o banco de dados, mantidas fora do controle de versão por questões de segurança. A cada implantação, o comando de *build* executa automaticamente a geração do cliente Prisma e a aplicação das migrações pendentes do banco de dados por meio do comando `prisma migrate deploy`, garantindo que o esquema em produção permaneça sempre sincronizado com o modelo de dados do projeto. Assim como o banco de dados, o plano gratuito do Render também suspende a aplicação após um período de inatividade, fazendo com que a primeira requisição subsequente acorde o *backend* e, de forma encadeada, o banco de dados, podendo levar dezenas de segundos para ser concluída. Para absorver essa latência combinada sem comprometer a experiência do usuário, a tela inicial do aplicativo passou a repetir a verificação de disponibilidade do servidor durante o carregamento, só liberando o acesso às demais telas quando o *backend* responde de fato, em vez de presumir que uma única tentativa seria suficiente.

4.7.3 Geração e Distribuição do Aplicativo

Para a geração do instalável Android, foi utilizado o EAS *Build*, serviço de compilação em nuvem do Expo Application Services (EXPO, 2025). Esse serviço elimina a necessidade de um ambiente local configurado com o Android SDK para gerar o pacote de instalação, compilando a aplicação em servidores remotos a partir do código-fonte do repositório. Os perfis de compilação foram definidos no arquivo `eas.json`, sendo o perfil `preview` utilizado para gerar um *Android Application Package* (APK) de distribuição interna, assinado e pronto para ser instalado diretamente no dispositivo do usuário, sem a necessidade de publicação em uma loja de aplicativos. Essa abordagem é compatível com a proposta deste trabalho de disponibilizar o Focus como um produto gratuito e de

código aberto, bastando o compartilhamento do arquivo APK gerado para que qualquer usuário com um dispositivo Android possa instalar e utilizar o aplicativo.

O instalável gerado para a versão final deste trabalho está disponível publicamente em <https://expo.dev/accounts/h0lzz/projects/focus/builds/f8e69283-1e75-4936-9490-7cdfc346665a>, de onde o arquivo APK pode ser baixado diretamente em um dispositivo Android. Como o aplicativo não é distribuído pela *Google Play Store*, o Google Play Protect, mecanismo de verificação de aplicativos nativo do Android, tende a bloquear a instalação de um instalável de origem externa por padrão. Para contornar esse bloqueio, é necessário desativar temporariamente a varredura do Play Protect (em *Play Store* → ícone de perfil → *Play Protect* → ícone de configurações → desativar a verificação de aplicativos) e, no aviso exibido durante a instalação, selecionar a opção equivalente a "Instalar mesmo assim"para prosseguir.

5 Considerações Finais

5.1 Conclusão

Ao longo do trabalho foram aplicados conceitos fundamentais da engenharia de *software*, desde o levantamento da literatura e a definição de requisitos até a modelagem, a arquitetura, a implementação e os testes do aplicativo. Conforme dito anteriormente, o objetivo do projeto foi o desenvolvimento de um aplicativo *mobile*, voltado à organização da rotina de estudos de estudantes universitários, por meio da integração do cronômetro Pomodoro configurável, *flashcards* com repetição espaçada organizados por disciplina, gamificação e *check-ins* de humor em uma única plataforma.

O levantamento da literatura realizado, com consultas na base científica Scopus, fundamenta a seleção das funcionalidades implementadas com base em evidências acadêmicas sobre gestão do tempo, repetição espaçada, gamificação e aprendizagem autorregulada; essa fundamentação teórica, no entanto, embasou a escolha das funcionalidades, e não substitui a validação de sua usabilidade e efetividade junto a usuários reais, que permanece como trabalho futuro. A literatura consultada indica que a gestão do tempo e a autoavaliação estão entre os fatores mais associados ao desempenho acadêmico, o que motivou a priorização do cronômetro Pomodoro e do algoritmo de repetição espaçada. Da mesma forma, os estudos sobre gamificação e aprendizagem autorregulada sustentam, respectivamente, os elementos de engajamento e os *check-ins* de bem-estar emocional como recursos relevantes, ainda que a gamificação, conforme discutido na Seção 2.5, tenda a impactar sobretudo a motivação do estudante.

A arquitetura em camadas implementada, com *backend* em Node.js e Express, *frontend* em React Native com Expo e banco de dados relacional PostgreSQL, foi projetada com o objetivo de favorecer a escalabilidade, a manutenibilidade e a conformidade com a LGPD, propriedades que foram reforçadas pelas decisões tomadas durante a implementação e os testes do sistema. A condução do *framework Lean Inception*, com a participação de duas proto-personas e do autor, contribuiu para explorar e organizar hipóteses sobre as necessidades dos usuários, resultando em um *Canvas* MVP e em um sequenciador de funcionalidades que orientaram o desenvolvimento incremental; por se tratar de uma amostra reduzida de participantes, essas hipóteses ainda carecem de validação junto a uma base maior de usuários reais. A aplicação combinada das metodologias *Scrum*, Kanban, XP e TDD apoiou um ciclo de desenvolvimento ágil, rastreável e orientado à qualidade.

Portanto, este trabalho entrega um aplicativo funcional, teoricamente fundamentado e centrado no usuário, resultado da implementação completa do escopo de MVP

definido durante o *Lean Inception* e rastreável até os requisitos funcionais estabelecidos.

5.2 Trabalhos Futuros

Diante dos resultados alcançados, com a implementação completa das funcionalidades priorizadas no Sequenciador e no *Canvas* MVP, torna-se viável a continuidade do desenvolvimento do projeto por meio da incorporação dos incrementos identificados durante o *Lean Inception*, mas não priorizados nesta primeira versão. Por se tratar de um projeto gratuito e *open source*, com o código-fonte distribuído sob a licença MIT nos repositórios <https://github.com/H0lzz/focus-backend> e <https://github.com/H0lzz/focus-mobile>, incrementos futuros podem ser propostos e implementados por outros desenvolvedores interessados em contribuir com o aplicativo.

Como trabalhos futuros, destacam-se:

- a validação das proto-personas e das hipóteses de necessidade levantadas neste trabalho junto a uma amostra maior de usuários reais, por meio de entrevistas, testes de usabilidade ou pesquisas de campo;
- a implementação de um modo *offline-first*, com armazenamento local e sincronização posterior dos dados, funcionalidade identificada como desejável durante o *Lean Inception*, mas classificada fora do escopo do MVP em razão de seu esforço técnico e da baixa prioridade diante do público universitário do produto, que tende a ter acesso mais estável à internet;
- a incorporação de Inteligência Artificial para a geração automatizada de *flashcards* a partir de materiais enviados pelo usuário, como PDFs e imagens de anotações, funcionalidade que não faz parte do escopo do MVP, mas que poderia representar um diferencial relevante;
- a automação de testes de integração e de sistema para as rotas HTTP e a camada de controladores do *backend*, complementando a cobertura já obtida pelos testes unitários da camada de serviços;
- a implementação das demais funcionalidades classificadas como incrementos no sequenciador, não contempladas nesta versão do MVP.

Referências

AMGHAR, A.; BENCHEKROUN, B. A. A comprehensive framework for higher education 4.0. *Multidisciplinary Reviews*, 2025. Disponível em: https://doi.org/10.31893/multirev.2026193. Acesso em: 23 jun. 2026. Citado na página 23.

ANKI. *Anki - powerful, intelligent flashcards*. 2026. Acesso em: 25 ago. 2026. Disponível em: <https://apps.ankiweb.net/>. Citado 2 vezes nas páginas 27 e 28.

ANTHROPIC. *Claude Code*. 2026. Acesso em: 05 nov. 2026. Disponível em: <https://claude.com/claude-code>. Citado na página 70.

APROVADO. *Aprovado - Controle de Estudos*. 2026. Acesso em: 23 jun. 2026. Disponível em: <https://aprovadoapp.com/pt>. Citado 2 vezes nas páginas 25 e 26.

BICKERDIKE, A. et al. Learning strategies, study habits and social networking activity of undergraduate medical students. *International Journal of Medical Education*, v. 7, p. 230–236, 2016. Citado na página 13.

BRASIL. *Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD)*. 2018. Acesso em: 23 jun. 2026. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm>. Citado 2 vezes nas páginas 32 e 72.

BROEK, G. S. E. V. D. et al. Gamified feedback in adaptive retrieval practice: Points and progress-bars enhance motivation but not learning. *Computers in Human Behavior*, v. 177, p. 108862, 2026. Citado na página 24.

CAROLI, P. *Lean Inception: Como alinhar pessoas e construir o produto certo*. São Paulo: Caroli.org, 2017. Citado 6 vezes nas páginas 46, 47, 48, 55, 56 e 59.

CASSIERI, P. et al. On the use of test-driven development for embedded systems. *Information and Software Technology*, v. 187, p. 107779, 2025. Citado na página 42.

CHERNEV, A.; BÖCKENHOLT, U.; GOODMAN, J. Choice overload: A conceptual review and meta-analysis. *Journal of Consumer Psychology*, v. 25, n. 2, p. 333–358, 2015. Citado na página 35.

CIRILLO, F. *The Pomodoro Technique: The Life-Changing Time-Management System*. New York: Currency, 2018. Citado na página 33.

DRAW.IO. *Draw.io Documentation*. 2025. Acesso em: 23 jun. 2026. Disponível em: <https://www.drawio.com/docs/>. Citado na página 68.

EBBES, R. et al. Promoting self-regulated learning during the covid-mandated remote learning period: Insights from interviews with primary school teachers. *Teaching and Teacher Education*, v. 169, p. 105287, 2026. Citado na página 34.

EXPO. *Expo Documentation*. 2025. Acesso em: 23 jun. 2026. Disponível em: <https://docs.expo.dev/>. Citado 2 vezes nas páginas 69 e 80.

FERNáNDEZ, M. S. et al. Metodologías didácticas utilizadas en las escuelas de formación policial para enseñar derechos humanos. *European Public & Social Innovation Review*, v. 11, p. 01–21, 2026. Citado na página 23.

FERREIRA, B. et al. Investigating problem definition and end-user involvement in agile projects that use lean inceptions. In: *International Conference on Information and Software Technologies*. [S.l.: s.n.], 2024. Citado 2 vezes nas páginas 45 e 46.

FIGMA. *Figma Learn*. 2025. Acesso em: 23 jun. 2026. Disponível em: <https://help.figma.com/hc/pt-br>. Citado na página 68.

FU, Y. et al. Unlocking academic success: the impact of time management on college students' study engagement. *BMC Psychology*, v. 13, n. 323, 2025. Citado na página 22.

GITHUB. *About Git*. 2025. Acesso em: 23 jun. 2026. Disponível em: <https://docs.github.com/en/get-started/using-git/about-git>. Citado na página 69.

GOBOV, D.; ZUIEVA, O. Software quality attributes in requirements engineering. *International Journal of Information Technology and Computer Science (IJITCS)*, v. 17, n. 4, p. 38–48, 2025. Citado 4 vezes nas páginas 66, 72, 73 e 74.

GOERICKE, S. (Ed.). *The Future of Software Quality Assurance*. Cham: Springer Nature Switzerland, 2020. ISBN 978-3-030-29509-7. Citado 2 vezes nas páginas 77 e 78.

HABITICA. *Habitica - Gamify Your Life*. 2026. Acesso em: 25 ago. 2026. Disponível em: <https://habitica.com/static/home>. Citado 2 vezes nas páginas 28 e 29.

HARVEY, C.-J. et al. Improving medical students' learning strategies, management of workload and wellbeing: a mixed methods case study in undergraduate medical education. *BMC Medical Education*, v. 25, n. 606, 2025. Citado na página 23.

HSU, T.-C.; HSU, T.-P. Effects of game-based learning integrated with different thinking-guided methods on computational thinking of elementary school students. *Thinking Skills and Creativity*, v. 60, p. 102056, 2026. Citado na página 24.

IBARRA-TORRES, F.; URBIETA, M.; MEDINA-MEDINA, N. Applying scrum to knowledge transfer among software developers. *International Journal of Software Engineering and Knowledge Engineering*, v. 35, n. 9, p. 1239–1265, 2025. Citado na página 43.

KHALID, S. et al. Ensuring quality in software requirement engineering process: A comparative study. *Egyptian Informatics Journal*, v. 31, p. 100754, 2025. Citado 4 vezes nas páginas 59, 71, 73 e 74.

KHALIL, M. K.; WILLIAMS, S. E.; HAWKINS, H. G. Learning and study strategies correlate with medical students' performance in anatomical sciences. *The Anatomical Record*, p. 1–7, 2024. Citado na página 14.

KIDMAN, P. G. et al. When and why adults abandon lifestyle behavior and mental health mobile apps: Scoping review. *Journal of Medical Internet Research*, v. 26, p. e56897, 2024. Citado na página 34.

KIM, J.; KOOK, J. A research on the impact of a class diagram layout and complexity on system model understandability. In: *International Conference on Research in Adaptive and Convergent Systems (RACS), 2024, Pompei*. New York: ACM, 2024. Citado na página 73.

KLEIJN, W. C.; PLOEG, H. M. van der; TOPMAN, R. M. Cognition, study habits, test anxiety, and academic performance. *Psychological Reports*, v. 75, n. 3, p. 1219–1226, 1994. Citado 2 vezes nas páginas 12 e 13.

KOKI. *Koki FlashCards*. 2026. Acesso em: 5 set. 2026. Disponível em: <https://anki.edu.vn/>. Citado 2 vezes nas páginas 29 e 30.

KOSE, T.; MEDE, E. Investigating the use of a mobile flashcard application rememba on the vocabulary development and motivation of EFL learners. *MEXTESOL Journal*, v. 42, n. 4, p. 1–26, 2018. Citado na página 32.

LETAW, L. *Handbook of Software Engineering Methods*. 2. ed. Corvallis: Oregon State University, 2024. Acesso em: 23 jun. 2026. Disponível em: <https://open.oregonstate.education/setextbook>. Citado na página 78.

LIU, H.; LU, X.; YAN, Y. Exploring the mediating role of anxiety between resilience and academic achievement in students' English learning. *Humanities & Social Sciences Communications*, v. 12, n. 1, p. 1773, 2025. Citado 3 vezes nas páginas 12, 14 e 35.

LO, C.-M.; HUANG, S.-J. Applying model-driven approach to building rapid distributed data services. *IEICE Transactions on Information and Systems*, E95-D, n. 12, p. 2796–2809, 2012. Citado na página 77.

LOLI, N. L. T. et al. Efecto del método kanban en el aprendizaje basado en problemas aplicado a estudiantes universitarios. *Revista De Ciencias Sociales*, XXXI, n. 3, p. 520–541, 2025. Citado na página 44.

MARENGO, A. et al. Research AI: integrating AI and gamification in higher education for e-learning optimization and soft skills assessment through a cross-study synthesis. *Frontiers in Computer Science*, v. 7, p. 1587040, 2025. Citado 2 vezes nas páginas 13 e 15.

NEON. *Neon Docs*. 2026. Acesso em: 06 out. 2026. Disponível em: <https://neon.tech/docs/introduction>. Citado na página 79.

NIELSEN, J. *Usability Engineering*. San Francisco: Morgan Kaufmann, 1994. Citado na página 32.

OGUT, E. Assessing the efficacy of the pomodoro technique in enhancing anatomy lesson retention during study sessions: a scoping review. *BMC Medical Education*, v. 25, p. 1440, 2025. Citado na página 33.

POMPEO, D. D.; TUCCI, M.; HOORN, A. v. Editorial of the special issue on quality in software architecture. *The Journal of Systems and Software*, v. 230, p. 112602, 2025. Citado 2 vezes nas páginas 71 e 72.

RENDER. *Render Docs*. 2026. Acesso em: 06 out. 2026. Disponível em: <https://render.com/docs>. Citado na página 80.

SANTOS, M. F.; FILIPE, R. A.; CUNHA, J. C. From traditional to agile methodologies in software project management education: A case study. In: *International Conference on Education Technology and Computers (ICETC 2024)*. Porto, Portugal: ACM, 2024. p. 7. Citado 2 vezes nas páginas 40 e 41.

SASMITO, G. W. et al. Implementation of extreme programming method in developing website of agribusiness harvest transportation order data management. *Journal of Information Processing Systems*, v. 21, n. 2, p. 152–162, 2025. Citado na página 41.

SHIN, S.-S. et al. Difficulties in understanding the transition between database design stages: An experiment from a semantic distance perspective. *Journal of Engineering Education*, v. 114, n. 4, p. e70038, 2025. Citado na página 76.

SOMMERVILLE, I. *Software Engineering*. 9. ed. [S.l.]: Pearson, 2011. ISBN 9780137035151. Citado na página 66.

STUDYTOK. *StudyTok*. 2026. Acesso em: 23 jun. 2026. Disponível em: <https://studytok.com/>. Citado 2 vezes nas páginas 26 e 27.

TESTDRIVEN. *Test-Driven Development*. 2025. Acesso em: 23 jun. 2026. Disponível em: <https://testdriven.io/test-driven-development/>. Citado na página 42.

WANG, L.; DU, Z. Intelligent technology-driven hybrid english classroom model for enhanced personalization and learning efficiency. *Journal of Circuits, Systems, and Computers*, v. 35, n. 1, p. 2550372, 2026. Citado na página 23.

WOZNIAK, P. A.; GORZELANCZYK, E. J. Optimization of repetition spacing in the practice of learning. *Acta Neurobiologiae Experimentalis*, v. 54, p. 59–62, 1994. Citado na página 33.

ZHOU, Y.; GRAHAM, L.; WEST, C. The relationship between study strategies and academic performance. *International Journal of Medical Education*, v. 7, p. 324–332, 2016. Citado 4 vezes nas páginas 12, 13, 14 e 22.